

Readerboard and Busylight User's Guide

Includes details on assembly, installation, and programming these messaging units.



The information in this document, and the hardware and software it describes, are hobbyist works created as an educational exercise and as a matter of personal interest for recreational purposes.

It is not to be considered an industrial-grade or retail-worthy product. It is assumed that the user has the necessary understanding and skill to use it appropriately. The author makes NO representation as to suitability or fitness for any purpose whatsoever, and disclaims any and all liability or warranty to the full extent permitted by applicable law. It is explicitly not designed for use where the safety of persons, animals, property, or anything of real value depends on the correct operation of the software.

As an educational exercise, it is assumed that the user has the necessary skill to appropriately set up, build, and use this project. Do not proceed to use, whether exactly as provided in the repository or with modifications of your own, unless you know exactly what you're doing and have verified that your actions and the product materials are suitable for your purpose and function as expected.

For Readerboard hardware (PCB) version 3.4.2, Busylight hardware version 2.2.1 (light tree version 1.1), and firmware version 2.3.8.

(Experimental support for Adafruit 16×32, 32×32, 16×32×2, and 64×64 RGB matrix displays is a work in progress with current PCB revision 0.0.0.)

Copyright © 2023, 2024 by Steven L. Willoughby (aka MadScienceZone), Aloha, Oregon, USA. All Rights Reserved. This document is released under the terms and conditions of the Creative Commons “Attribution-NoDerivs 3.0 Unported” license. In summary, you are free to use, reproduce, and redistribute this document provided you give full attribution to its author and do not alter it or create derivative works from it. See

<http://creativecommons.org/licenses/bynd/3.0/>

for the full set of licensing terms.



CONTENTS

Contents	iii
List of Figures	vi
List of Tables	vi
1 Hardware	1
Version 1	1
Version 2	1
Version 2	2
Version 3	2
Firmware Implications	3
Construction	3
Busylight Devices	6
Controller for Adafruit Matrices	9
2 Protocol Description	13
Web Service API	13
Direct Host-to-Device Communication	14
Physical Layer	15
Transport Layer	15
Application Layer	19
Command Summary	19
%T@—Run Full Lamp Test Pattern	20
*—Strobe Lights in Sequence	20
<—Scroll Text Across Display	20
=—Set Operational Parameters	22
=#—Set Serial Number	23
=*—Display POST Banners	24
=&—Persist Settings in EEPROM	24
?—Query Discrete LED Status	24
@—Set Column Cursor Position	26
A—Select Font	26
B—Beep/Play Sound Sequence	27
C—Clear Matrix Display	29

D—Set Dimmer Levels	29
F—Flash Lights in Sequence	31
H—Draw Bar Graph Data Point	32
I—Draw Bitmap Image	33
K—Set Current Color	34
L—Light Multiple LEDs	34
Q—Query Readerboard Status	35
M—Morse Code	37
S—Light Single LED	38
T—Display Text	38
V—Manipulate Segmented LED Display	39
X—Turn off Discrete LEDs	41
3 Server-Only Commands	43
current—Report Current Display State	43
post—Post a Message to the Displays	44
postlist—Report of Message Queue	46
unpost—Remove a Posted Message	46
update—Update Dynamic Content	47
4 Pinouts	49
8p8c RS-485 Connectors	49
DIN-8 Busylight Tree Connectors	50
Connectors for Version 3.4 Boards	51
Adafruit Matrix Versions	53
Connectors for Legacy Boards	54
5 Installing and Using the Readerboard and Busylight Units	57
Power	57
Standalone Units	58
Networked Units	58
Compiling the Supporting Software	58
Setting up rbserver	58
Setting up Per-User Utilities	58
Diagnostic Displays	58
6 Font Glyphs	59
Font #0 standard.font	59
Font #1 standard_variable.font	68
Font #2 symbol.font	77
Font #3 double.font	85
Font #4 led16.font	89
Font #5 led16a.font	92
Default Glyph	94
Font Definition Files	94

7 Preparing and Updating Firmware Images

97

8 Schematics and Diagrams

103

LIST OF FIGURES

2.1	14-Segment LED Alphanumeric Display	40
4.1	Female 8p8c Modular Jack with T-568B Color Codes	50
4.2	DIN-8 Jack (looking into socket from front)	51

LIST OF TABLES

1.1	Bill of Materials	4
1.2	Bill of Materials (Busylight)	8
1.3	Bill of Materials for AdaSheild Board	10
1.4	Bill of Materials for AdaSheild Board	11
2.1	Examples of RS-485 Escape Bytes.	19
2.2	Summary of All Commands	21
2.3	Control Codes in String Values	22
2.4	Baud Rate Codes	23
2.5	ASCII Encoded Integer Values (0–63)	26
2.6	Font Codes	27
2.7	Font Table for Fonts #0 and #1	28
2.8	Font Table for Font #2	29
2.9	Font Table for Font #3	30
2.10	Discrete LED Codes and Colors	31
2.11	Transition Effect Codes	34
2.12	Color codes for <i>(rgb)</i> parameters	35
2.13	Alignment Codes	39
3.1	Transition codes for use in post command	44

<i>List of Tables</i>	vii
-----------------------	-----

3.2 Alignment codes for use in post command	44
3.3 Color codes for use with the post command	45
3.4 Special Codes in Post Messages	46
7.1 Firmware Configuration Values	98

HARDWARE

People who are really serious
about software should make
their own hardware.

—Alan Kay

AS A HOBBY PROJECT, the design of the hardware and software for the read-board project “grew in the telling” as new ideas sprang to mind. As of this writing, there are three different models of hardware prototypes which were designed and created.

Version 1

The initial prototype (PCB 1.0.1) was a fairly large board, approximately $20\frac{5}{8} \times 6\frac{3}{8}$ ” with an LED pitch of $\frac{1}{8}$ ”.

It requires an Arduino Mega 2560 or Arduino Due to drive it. A custom-made shield board attaches to the Arduino, which provides connectors for power and ribbon cables to drive the display (one to control the matrix, the other for the 8 status LEDs which appear in a horizontal row along the bottom right of the matrix).

Version 2

This version was based on the 1.0 design, but shrunk to a board size of about $18\frac{3}{16} \times 5\frac{3}{16}$ ” with an LED pitch of $\frac{1}{4}$ ”. It also replaced the obsolete TPIC6B595 with the newer STPIC6D595 chip. Further, it relocated the ribbon cable which

connects the Arduino controller to the LED matrix, moved the eight status LEDs to a vertical column to the right of the matrix, and switched to resistor arrays for the matrix current limiting resistors, instead of the individual discrete resistors used on the 1.0.1 board.

Version 2

Version 2.1 was a significant change to the 2.0 board in terms of how it interfaces with the Arduino, although it had the same size and physical layout as the 2.0 board.

This one eliminates the need for the ribbon cables and shield board. Instead, the Arduino controller mounts directly to the back of the display board. This means that the lower portion of the left edge of the enclosure has all of the external connections in one place, including power, RS-485 in and out, and the USB connector(s) on the Arduino itself.

Version 3

Revision 3.x of the hardware expands on the version 2.1 design by replacing the single-color LEDs with RGB LEDs arranged into 24 logical rows—each of the eight physical rows is logically a row of red, green, and blue LED elements.

Note that the discrete status LEDs are mislabeled on the revision 3.2.2 PCB. L_0 should be the LED on the bottom (canonically green) and L_7 is on top (white).

Version 3.4.1 (current model)

Version 3.4 added support for playing sounds on a speaker and added a diode (D520) to prevent the Arduino from trying to power the LEDs by itself if it is powered from the USB port but the external 9 V supply is not active.

Version 3.3.1

Version 3.3 of the hardware corrects some minor issues with the 3.2.2 revision by adding a cutout to avoid colliding with the connectors on the Arduino board and adjusting the mounting holes slightly. It also corrects the mislabeled L_0 – L_7 LEDs

Additionally, it adds R51 for Arduino Due reset circuits, J3 to manually reset the Arduino, and J4, R52, R53, and U16 to support the addition of a external EEPROM chip for microcontrollers (such as the Due) which don't have one onboard.

This was further improved with revision 3.3.1 (the current version), which replaced terminal block J1 with a higher-density terminal which

allows easier interconnection of two CAT6 cables without the need for off-board splices. The power supply now connects to a new terminal block J5. The power to the RS-485 transceiver chip is also now switchable to either 3.3 V or 5 V so it can be appropriately matched to the operating voltage of the Arduino board without damaging the Arduino's input ports.

Firmware Implications

The boards before PCB revision 3.3.0 are not supported by the firmware as currently stored in the project repository. They were prototype ideas which were never built, so this shouldn't be a problem.

There are compile-time switches documented starting on p. 97 which adjust the firmware image for different microcontrollers, default settings—or the only settings if your hardware does not include an EEPROM memory—and possibly, in the future, different board revisions.

Construction

Once you have created a printed circuit board (PCB) from the fabrication files included with the readerboard Git repository and obtained the parts as listed in Figure 1.1, carefully solder the components in place.

Warning! Danger! Only perform the assembly operation if you are qualified to do so. Soldering electronic components may be hazardous due to the high temperatures, sharp objects, electrical voltages, potentially toxic materials, and other dangers. This requires skill and expertise as well as personal protection equipment to perform safely. Please also observe all precautions to avoid damaging the electronic components themselves from harm due to heat or static electricity discharge.

Suggested Order

For best results, solder components in a way that allows best access to the solder points without some parts getting in the way of connecting subsequent ones. For example:

1. Solder all the surface mount chips first: U0–U15 (and optionally, U16).
 - U0, U12, U13: CD74HCT238PWR
 - U1, U2, U14, U15: ULN2803CDWR (ULN2803A SOP-18 packages may be used if aligned to the top 18 pads, leaving the bottom two empty, if they have otherwise identical pinouts and characteristics)
 - U3–U10: STPIC6D595MTR

Part No.	Qty	Mouser	Description
	1		Printed circuit board rev 3.2.2
	1		Arduino Due or Mega 2560
C0–9,11,12	12	810-FA18X7R1H10400	0.1 μ F capacitor
C10	1	667-ECA-1HM100I	10 μ F electrolytic capacitor
C13	1		100 μ F electrolytic capacitor
D0–511	512	[1]	LED, 5mm, RGB, common cathode
D512	1		LED, 5mm, green
D513–14	2		LED, 5mm, amber (yellow)
D515–16	2		LED, 5mm, red
D517–18	2		LED, 5mm, blue
D519	1		LED, 5mm, white
D520	1		power diode, 4 A
J0	1		Arduino stacking pin set
J1a,b	2	179-TBLH10-381-08BK	8-position terminal block
J2,3	2	649-69190-102HLF	2-position jumper header
J4	1		3-position jumper header
J5,6	2		2-position Euro-style terminal block
P0–2	3		2-position jumper shunt
Q0–23	24	942-IRF9530NPBF	IRF9530 p-channel MOSFET
Q24	1		2N2222 or BC547 NPN transistor
R0–7,35–51	24	604-MFR-24FRF521K	1K resistor
R8–23	16		4 $\times$ 360 Ω resistor array, isolated
R24,29–31	4	603-MFR-25FTE52-300R	300 Ω resistor
R25–28	4		360 Ω resistor
R32–34,52–54	6	603-MFR-25FRF5210K	10K resistor
R _{TERM}	1		120 Ω resistor
U0,12,13	3	595-CD74HCT238PWR	CD74HCT238PWR non-inverting 3-to-8 decoder
U1,2,14,15	4		ULN2803CDWR octal NPN Darlington array
U3–10	8		STPIC6D595MTR power 8-bit shift register
U11	1	595-THVD1439VDR	THVD1439 half-duplex RS-485 driver/receiver
U16	1	579-24LC256T-E/SN	AT24C256 EEPROM
	24		TO-220 heat sinks for Q0–23 (optional)
	1		9V 2A DC power supply

[1] I found these at Amazon.com: [EDGELEC 5mm RGB Tri-Color 4Pin LED Diodes Common Cathode](#).

Table 1.1: Bill of Materials

- U11: THVD1439DR (Note that this chip is oriented 180° from the other chips on the board, with pin 1 to the lower right.)
 - If needed for your microcontroller (Arduino Due, usually), solder U16: AT24C256 EEPROM chip.
2. Solder the discrete resistors: R0–R7, R24–R51, and R54 (and optionally, R52 and R53). See below first to ensure the correct values are used for R24–R31 based on your choice of corresponding LEDs.
 - R0–R7, R35–50: 1K
 - R24, R29–R31: 300Ω
 - R25–R28: 360Ω
 - R32–R34: 10K
 - If U16 is installed, solder in R52–R53: 10K
 - If using sound output, solder in R54: 10K
 3. Solder the 0.1 μF capacitors: C0–C9, C11–C12.
 4. Solder the resistor arrays: R8–R23.
 5. Solder the RGB LEDs D0–D511. Ensure that the common cathode goes to pin 2 so that pins 1, 3, and 4 are respectively the red, green, and blue anodes.
 6. Solder the single-color LEDs D512–D519 in the colors of your choice, being mindful of the polarity. The anode has a square pin on the PCB. Our recommendation:
 - D512 (bottom): green (with R24=300Ω)¹
 - D513: amber² (with R25=360Ω)
 - D514: amber (with R26=360Ω)
 - D515: red (with R27=360Ω)
 - D516: red (with R28=360Ω)
 - D517: blue (with R29=300Ω)
 - D518: blue (with R30=300Ω)
 - D519 (top): white (with R31=300Ω)
 7. **If you are using an Arduino Due:** solder R51 (1K).

¹Resistor values based on typical LED voltage and current ratings. Check to see what is appropriate for your components.

²In deference to the effort my high school drama teacher went through to get us to refer to any yellow stage lights and gels as “amber” and the preference to using “amber” for signal lamp colors in other contexts and regions, both terms will be used interchangeably in this project.

8. **If you are using an Arduino Mega 2560:** Solder the 10 μ F capacitor C10, being mindful of the correct polarity. **Do not solder both R51 and C10. Only one or the other will be appropriate for a given microcontroller.**
9. Solder the Arduino bus stacking pins (six connectors—1 $\times$ 10-pin, 5 $\times$ 8-pin—all labeled collectively as “J0”). Ensure the pins extend down and out the back of the board.
10. Solder the 2-pin headers J2 and J3, and the 3-pin header J4.
11. Solder the terminal blocks J1 (which is actually a pair of 8-pin terminal blocks J1a and J1b), and J5.

Option: Using Single-Color LEDs

If you don’t need RGB LEDs for your application, the same PCB may be used with single-color 5mm LEDs instead. Simply solder them onto the board so that their cathode is on pin 2 and the anode is on pin 4 of the LED footprint. This means the LEDs will electrically all be connected to the blue circuit.

Since they won’t be used, omit parts U0, U2, U12, U14, C0, C11, Q0–Q15, R0–R7, and R35–R42.

Set the firmware value `HW_MODEL` to `MODEL_3xx_MONOCHROME` and recompile the firmware. This changes the behavior of the controller so that it understands it can only drive one color which is attached to the blue color plane. Now, all colors (other than “off”) will turn the LEDs on.

Busylight Devices

The readerboard units are designed to be compatible with another project by the author. The “Busylight” is a small stand-alone indicator which can be used to indicate simple statuses like room occupancy or whether someone is busy, free, on a phone call, etc.

The busylight can drive up to seven LEDs. If directly connected to a host computer via USB, it can be powered entirely from the USB port. Starting with version 2, it can also receive commands via RS-485 network. In that case it can be powered by a 9 V supply over the same CAT5, CAT5e, or CAT6 cable carrying the serial communications.

Firmware

Starting with revision 2 of the Busylight, it is driven by a firmware image compiled from the same source code as the Readerboard units. Setting the compiler preprocessor symbol `HW_MODEL` to `MODEL_BUSYLIGHT_2` will cause the Arduino compiler to prepare the image for a Busylight, excluding the

code that drives the readerboard-specific functions. In such firmware images, `HW_MC` should be set to `HW_MC_PRO` since the Busylight is based on the SparkFun Arduino-compatible Pro Micro microcontroller.

Also set the `SERIAL_VERSION_STAMP` symbol to reflect the device serial number, hardware and firmware revision numbers. See Chapter 7 starting on p. 97 for more information about preparing firmware images.

Construction

Once you have created a printed circuit board (PCB) from the fabrication files included with the Git repository and obtained the parts as listed in Figure 1.2, carefully solder the components in place.

Warning! Danger! Only perform the assembly operation if you are qualified to do so. Soldering electronic components may be hazardous due to the high temperatures, sharp objects, electrical voltages, potentially toxic materials, and other dangers. This requires skill and expertise as well as personal protection equipment to perform safely. Please also observe all precautions to avoid damaging the electronic components themselves from harm due to heat or static electricity discharge.

Suggested Order

For best results, solder components in a way that allows best access to the solder points without some parts getting in the way of connecting subsequent ones. For example:

1. Solder the surface mount components:
 - U0 ULN2003A 7-circuit NPN Darlington transistor array. Note that the board was designed for narrow SOIC-16 packages but there is an additional column of pads for a wider version of this chip.
 - U1 THVD1439 RS-485 driver/receiver.
2. Solder through-hole components:
 - R0–R2 10K resistors
 - C0 0.1 μ F capacitor
3. Solder the sockets:
 - J0 8-pin DIN socket
 - J1 16-circuit Euro-style terminal block (alternatively you may choose to directly solder the CAT6 wires into the holes on the PCB.
 - Arduino stacking pins.
4. Place the microcontroller board onto the stacking pins.

Part	Qty	Description
	1	Printed circuit board rev 2.1
	7	LED tier printed circuit board rev 1.1
	1	SparkFun Pro Micro Arduino-compatible μC , 5 V version
C0	1	0.1 μF capacitor
C1	1	100 μF capacitor
D0	1	power diode, 1 A
J0	1	8-pin DIN connector, PCB mount
J1	1	16-pin Euro-style terminal block
J2	1	Arduino stacking pin set
J3	1	2-pin Euro-style terminal block
P0	1	8-pin DIN plug
Q0	1	BC547 or 2N2222 NPN transistor
R0–3	4	10K resistor
U0	1	595-ULN2003AIDR ULN2003A NPN Darlington array
U1	1	THVD1439 half-duplex RS-485 driver/receiver
	14	5-pin bussed resistor network (2/LED tier)*
	56	LED, 5 mm or 3 mm, through-hole (8/LED tier)†
	1	$\frac{1}{4}$ " screw rod
	1	Enclosure‡
	8	Nylon spacers between (and above and below) the tiers
	2	$\frac{1}{4}$ " hex nuts

*Choose each tier's pair of resistor arrays appropriately for the voltage drop for the LEDs to be used on that tier. Assuming a 5 V supply and typical LED characteristics for a desired 20 mA current draw, that would be 150Ω or larger for red and amber LEDs and 100Ω or larger for blue, green, and white LEDs. Check your LED datasheet to confirm. Examples: Mouser 652-4605X-1LF-100 (100Ω) or 652-4605X-1LF-150 (150Ω).

†Choose LED colors as desired for each tier (standard is green, 2 $\times$ amber, 2 $\times$ red, blue, and white).

‡One idea for an inexpensive enclosure is a clear plastic tube storage container with an inside diameter large enough to hold the LED tier boards.

Table 1.2: Bill of Materials (Busylight)

5. Build each LED tier of 8 LEDs:

- Solder eight 3 mm LEDs around the edge of the board with their leads straddling the edge, so the anode is on the side of the PCB labelled “anode side” and the cathode is on the “cathode side.” Solder the leads to the exposed pads and trim the leads so they don’t extend past the pads.
- Solder the two resistor arrays, making sure to select the correct values for the specific LEDs used on this tier board.

6. Assemble the tier stack by threading the tiers onto the $\frac{1}{4}$ " threaded rod with nylon spacers between them, secured in place with hex nuts on either end of the stack. The following assumes the use of a length of CAT6 cable between the PCB and the light tree stack.

- Solder one end of the cable to P0, the 8-bin DIN plug. Suggested color assignments:
 - Pin 1 (sixth tier/blue): blue
 - Pin 2 (third tier/amber): brown/white
 - Pin 3 (fifth tier/red): orange/white
 - Pin 4 (fourth tier/red): orange
 - Pin 5 (seventh tier/white): blue/white
 - Pin 6 (second tier/amber): brown
 - Pin 7 (+5 V supply): green
 - Pin 8 (bottom tier/green): green/white
- Strip the insulation from the green wire to the height of the assembled tier tree. As each tier is added, thread this bare wire through the “V+” hole of each tier, soldering it to each tier along the way.
- Attach the wire for each tier to its “GND” hole.
- Thread the remaining wires through the larger holes to pass them up to the next tiers.

Controller for Adafruit Matrices

We obtained a number of pre-made RGB LED matrix displays from Adafruit which have a similar (but not quite the same) control logic to our own design, which allows us to also control those boards using only a slight variation in our hardware and firmware design.

The hardware to drive them is a small shield board that plugs onto an Arduino Due or Mega 2560. It incorporates the same control circuitry as our original readerboard circuit described above, without the actual LEDs themselves or the discrete chips that drive the rows and columns of the display, since those are already on the pre-made displays.

This shield provides two ribbon cable connectors: J7, a 16-pin cable which sends the control signals to the matrix display, and J8, a 10-pin cable that can drive an 8-LED discrete status display such as is on the busylight and our original readerboard.

The following describes the assembly of one of these shield boards.

AdaShield/QS Variation

The author produced a variation on the AdaShield boards for another project, but they may still be useful for general use as an alternative to the above-described board in that it lacks a speaker output, but adds an I²C alphanumeric LED display output and expands the discrete LED output from 8 LEDs to 14.

Part	Qty	Mouser	Description
	1		AdaShield Printed circuit board rev 0.0.0
	1		Arduino Due or Mega 2560
C0	1		100 μ F electrolytic capacitor
C1	1	810-FA18X7R1H10400	0.1 μ F capacitor
C2	1	667-ECA-1HM100I	10 μ F electrolytic capacitor
D0	1		power diode, 1 A
J0	1		Arduino stacking pin set
J1a,b	2		8-position terminal block
J2,3	2		2-position jumper header
J4	1		3-position jumper header
J5,6	2		2-position Euro-style terminal block
J7	1		16-position IDC header
J8	1	517-89110-0101	10-position IDC header
P0–2	3		2-position jumper shunt
Q0	1		2N2222 or BC547 NPN transistor
R0–5	6	603-MFR-25FRF5210K	10K resistor
R6	1	604-MFR-24FRF521K	1K resistor
R _{TERM}	1		120 Ω resistor
U0	1		THVD1439 half-duplex RS-485 driver/receiver
U1	1		AT24C256 EEPROM
U2	1		ULN2803CDWR octal NPN Darlington array
	1		9V 2A DC power supply

Table 1.3: Bill of Materials for AdaSheild Board

Part	Qty	Mouser	Description
	1		AdaShield/QS Printed circuit board rev 0.0.0
	1		Arduino Due or Mega 2560
C0	1	667-ECA-1HM100I	10 μ F electrolytic capacitor
C1	1	810-FA18X7R1H10400	0.1 μ F capacitor
D0	1		power diode, 2 A
J0	1		Arduino stacking pin set
J1a,b	2		8-position terminal block
J2,3	2		2-position jumper header
J4	1		3-position jumper header
J5	1		2-position Euro-style terminal block
J6	1		10-position IDC header
J7,8	2		16-position IDC header
P0–2	3		2-position jumper shunt
R0–4	5	603-MFR-25FRF5210K	10K resistor
R5	1	604-MFR-24FRF521K	1K resistor
R6-8	3		Resistor array, isolated, 4-resistors*
R9,10	2		Resistor*
R _{TERM}	1		120 Ω resistor
U0	1		THVD1439 half-duplex RS-485 driver/receiver
U1	1		AT24C256 EEPROM
U2,3	2	595-ULN2003AIDR	ULN2003A NPN Darlington array
	1		9V 2A DC power supply

*R6–10 each limit current to the discrete LEDs. Choose their values appropriately for the supply voltage and current needs of the LEDs to be driven.

Table 1.4: Bill of Materials for AdaSheild Board

CHAPTER 2

PROTOCOL DESCRIPTION

I don't stand on protocol. Just call me
your Excellency.

—Henry Kissinger

THE CONTROL PROTOCOL used to display information on the readerboard sign is very simple. Commands are expressed largely in plain ASCII characters and are executed immediately as they are received.¹

In addition to plain, printable 7-bit ASCII characters, a few control codes are recognized as described below. String data may include any 8-bit value except as otherwise indicated.

Web Service API

The organization of the rest of this chapter is oriented toward a description of the protocol used to control the hardware directly over a USB or RS-485 connection. However, you can set up a central server which is connected to all the readerboard signs and busylight indicators in a small area (via USB and/or RS-485). This allows clients to send commands to the server over the LAN for control of the readerboards.

The protocol between clients and servers uses messages which are simple HTTP requests posted to the server's listening port URL. If the server were configured to run on `example.org` on port 43210, then all client requests to that server would begin with

`http://example.org:43210/readerboard/v1/<message>?a=<targets>`

¹Technically, they may even be executed *while* they are being received.

Message Data Fields

As a general rule, if a message expects a field that is not provided, where possible, a suitable “zero” value will be assumed for that field. Any extra fields sent that were not expected are silently ignored.

Boolean fields may have values true or false; if they are presented without a value, they are considered to be true, and if completely missing they are assumed to be false. Thus,

```
.../readerboard/v1/<message>?a=<targets>&status=true
```

is equivalent to

```
.../readerboard/v1/<message>?a=<targets>&status
```

and

```
.../readerboard/v1/<message>?a=<targets>&status=false
```

is equivalent to

```
.../readerboard/v1/<message>?a=<targets>
```

Message Target Addresses

Every message also has a field called “a” which is a comma-separated list of target readerboards to which the message applies.

So, for example, the message containing a=2 would cause the server to transmit the corresponding command to the readerboard with address 2 directly connected to it via USB, or would transmit to the RS-485 network the command starting with the hex byte 92_{16} . (Why that byte value is sent will make sense after you read the protocol description that follows.)

If multiple devices are targeted, e.g., with a message containing the field a=2,5,37 with address 15 (i.e., $0F_{16}$) configured as the global address ad_G , it will transmit to the RS-485 network a command starting with the hex byte sequence BF_{16} 03_{16} 02_{16} 05_{16} 25_{16} , or—if applicable—will send the message directly to any of devices 2, 5, and 37 which are directly connected via USB.

Direct Host-to-Device Communication

The host PC (which may or may not be running the server program) connects to one or more readerboard and/or busylight devices by direct wire. The remainder of this chapter describes the protocol used for this communication. If you consider the standard ISO OSI network protocol stack, there are three layers of concern for us here.

7	Application	Application	} Readerboard/Busylight
6	Presentation		
5	Session		
4	Transport	Transport (LTP)	} USB or Lumos-compatible RS-485 network
3	Network		
2	Data Link		
1	Physical	Physical	

Physical Layer

At the lowest level, the serial bits are encoded and are carried over wires using either a direct USB connection to the host or a wider-area RS-485 serial network. We don't need to worry about that level of the communication here.

It is worth noting, however, that the readerboard and busylight devices use half-duplex RS-485 interfaces, so when they are communicating over that network and are asked to send data back to the host, they will reverse the data direction of these layers until that data are transmitted, then reverts back to receiving on the data channel.

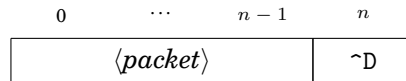
Transport Layer

This layer is responsible for getting the signals to the correct devices. This layer adds some data packaging around the message being sent to the device depending on the underlying physical connection type as described below.

USB Transport Protocol

Since each USB cable between the host and a device provides a single point-to-point connection, no addressing is needed (since there can only be one device on the other end of the wire, the host already knows which device it's talking to when sending messages on that USB port).

In this case, a single byte with value 04_{16} (character `^D`) is appended to the end of the data packet.



If there is an error parsing or executing a command, the device will ignore all subsequent input until a `^D` byte is received, whereupon it will

expect to see the start of another command. Thus, `^D` may not appear in any transmitted data except to terminate commands.

RS-485 Transport Protocol

The RS-485 standard is a differential serial signal which can be run up to 1,000 meters over unshielded twisted-pair cables such as CAT5 or CAT6. Up to 32 devices (including the host itself) can “tap into” this serial bus at various points (although physically this generally takes the form of “daisy chaining” connections where a cable plugs each device into the next one in the chain, with terminators plugged into the first and last devices).

Since multiple devices may exist on the same bus, they will all see the signals sent on that bus and thus must at least understand those messages enough to differentiate their messages from those destined for other devices even if they don’t know how to fully interpret other devices’ command sets.

To accomplish this, the RS-485 transport layer employed here uses a transport protocol the author designed for his Lumos light control devices. By being compatible with this standard, any set of readerboards, busy-lights, Lumos controllers, and any other devices following the same standard may be mixed on the same RS-485 network without conflict as long as they have distinct addresses assigned to them.

All data sent over RS-485 must conform to the following simple rules to be compatible with the Lumos Transport Protocol (hereafter LTP):

- The first byte in a command packet has its MSB set (1).
 - The lowest four bits of this byte specify the target device’s address (0–15).
 - The device may ignore this and all subsequent bytes if the address isn’t the global address or its own.
 - The remaining three bits specify the command to be executed on the target device.
 - Devices MAY implement extended commands in subsequent bytes to allow more than 8 commands, and/or respond to global target addresses by decoding additional target IDs in subsequent bytes to allow for multiple targets per device and/or device IDs greater than 15, but this is optional.
- All subsequent bytes in a command packet must have their MSB cleared (0).
- Devices MAY implement an escape code mechanism which does not otherwise violate the above rules to represent arbitrary 8-bit values in the data packet, which will be translated as the packet passes through the transport layer.

- Following this header, the remaining bytes of the packet are the application-layer command as documented later in this document. No trailing bytes are added to the end of the packet (unlike USB).

The specifics of how these rules translate to various packet headers are described in detail below.

Single Target or All Readerboards

To send a command to a single sign, the LTP packet header is a single byte encoded as:

7	6	5	4	3	2	1	0
1	0	0	1	$\langle ad \rangle$			

where $\langle ad \rangle$ is the sign's address on the bus, which must be a value in the range 0–15. This byte is followed by any command as described below. If the global address ad_G is given as the $\langle ad \rangle$ value, then all readerboards which have that set as their global address will obey the command.

Multiple Targets

Alternatively, a command may be targeted to multiple signs; in this case the LTP packet header is a multi-byte code:

7	6	5	4	3	2	1	0				
1	0	1	1	$\langle ad_G \rangle$							
0	0	$\langle n \rangle$									
0	0							$\langle ad_0 \rangle$			
$\vdots$											
								0	0	$\langle ad_{n-1} \rangle$	

where $\langle ad_G \rangle$ is the “global” device address which signals readerboards generally (see the = command below). This will send to the $\langle n \rangle$ devices addressed as $\langle ad_0 \rangle$ through $\langle ad_{n-1} \rangle$.

Note that device addresses are constrained to the range 0–15 if they are to be addressed in the initial LTP header start byte. However, using the multiple target header, device addresses in the range 0–63 may be used.

Response Headers

For commands which send a response back to the computer, the RS-485 response data stream begins with the LTP header

7	6	5	4	3	2	1	0
1	1	0	1	$\langle ad \rangle$			

where $\langle ad \rangle$ is the device address of the responding unit if $\langle ad \rangle < 16$. For devices with higher-numbered addresses, the following extended format is used instead:

7	6	5	4	3	2	1	0
1	1	1	1	$\langle ad_G \rangle$			
0	1						
0	0	$\langle ad \rangle$					

All Off

As a special case, the single byte

7	6	5	4	3	2	1	0
1	0	0	0	$\langle ad \rangle$			

will cause the readerboard addressed as $\langle ad \rangle$ to turn off all LEDs. If $\langle ad \rangle$ is the ad_G address, then all readerboards will turn off all LEDs.

No other command bytes need to follow; this byte is sufficient to turn off the sign(s). In other words, the LTP packet header itself is all that is sent—the command payload is empty.

The corresponding HTTP message is `/readerboard/v1/alloff?a= $\langle ad \rangle$`

Subsequent Command Bytes

All subsequent LTP bytes which follow the above binary headers *must* have their MSB cleared to 0.

To cover cases where a value sent as part of a command must have the MSB set, we use the following escape codes:

- A hex byte $7E_{16}$ causes the next byte received to have its MSB set upon receipt.
- A hex byte $7F_{16}$ causes the next byte to be accepted without any further interpretation.

Thus, the byte $C4_{16}$ is sent as the two-byte sequence $7E\ 44$, while a literal $7E$ is sent as $7F\ 7E$ and a literal $7F$ as $7F\ 7F$.

If there is an error parsing or executing a command, the readerboard will ignore all subsequent input until a byte arrives with its MSB set to 1, whereupon it will expect to see the start of another command.

A few illustrative examples are shown in Table 2.1.

Input Sequence	Resulting Byte
00	00
7D	7D
7F 7E	7E
7F 7F	7F
7E 00	80
7E 01	81
7E 7D	FD
7E 7E	FE
7E 7F	FF

Table 2.1: Examples of RS-485 Escape Bytes.

Application Layer

Once a command packet has been received and has been processed up through the transport layer, the remaining contents of the packet are interpreted by the target device as the command to be carried out. In the descriptions that follow, we will consider the matter of how the packet got to the device to have been fully resolved already so no mention will be made of the transport protocol. Assume the RS-485 header or USB trailer have been removed at this point.

Commands which contain arbitrary-size data fields, such as text strings or lists of LEDs to light in sequence, end such fields with a terminator byte. In most cases this may be either a dollar-sign (“\$”) character or the escape control character (hex byte 1B₁₆), indicated in the protocol diagrams below simply as “\$”. In cases where a dollar-sign could be part of the data, then only an escape character may be used to terminate the field, in which case the protocol description will show the terminator as “ESC”.

Command Summary

The eight discrete LEDs are intended for a simple display of status information in a manner analogous to the Busylight project by the same author.² To support this usage, the F, S, X, *, and ? commands are recognized in a manner compatible with how Busylight uses those same commands. These are categorized as “Busylight compatibility commands.” Unlike all other commands listed here, these are recognized regardless of case. Since the readerboard has a power supply capable of illuminating all of the status LEDs at once,³ a new command L is added which allows any arbitrary pattern of steady LEDs to be turned on.

²See github.com/MadScienceZone/busylight.

³The Busylight cannot, since it is powered from the host computer’s USB port.

The remaining commands are used for management of the matrix display. All commands are summarized in Table 2.2.

%T@—Run Full Lamp Test Pattern

0	1	2
%	T	@

/readerboard/v1/test?a=<ad>

Displays various test patterns on the display to show that each element of each LED is working correctly individually and in concert with others.

While running the test pattern set, the device is not responsive to other inputs or background operations.

*—Strobe Lights in Sequence

0	1	2	3	...	n	n + 1
*	<led ₀ >	<led ₁ >	<led ₂ >	...	<led _{n-1} >	\$

/readerboard/v1/strobe?a=<ad>&l=<led₀><led₁>...<led_{n-1}>

Each <led> value is an ASCII character corresponding to a discrete LED as shown in Table 2.10. An <led> value of “_” means there is no LED illuminated at that point in the sequence.

This command functions identically to the F command (see below), except that the lights are “strobed” (flashed very briefly with a pause between each light in the sequence).

<—Scroll Text Across Display

0	1	2	...	n+1	n+2
<	<loop>	<string>			ESC

0	1	2	3	4	5	...	n+4	n+5
<	@	<row>	<height>	<loop>	<string>		ESC	

/readerboard/v1/scroll?a=<ad>&loop=<bool>&t=<string>
&row=<row>&height=<height>

In the second form, the <row> and <height> parameters define a region of the display that extends to the full width of the display but includes <height> pixel rows starting at pixel #<row> (with row #0 being the topmost row of the device). In this case, the command behaves as though this region is the only part of the sign that exists as far as where characters will be drawn and

Message	Cmd	Description	Notes
strobe	*	Strobe LEDs in Sequence	[1]
busy	?	Query discrete LED status	[1] [2] [5]
flash	F	Flash LEDs in Sequence	[1]
light	L	Light one or more LEDs steady	[3]
light	S	Light one LED steady	[1]
off	X	All LEDs off	[1]
	^D	Abort/terminate command	
test	%T@	Full lamp test	
scroll	<	Scroll text across display	
configure-device	=	Set operational parameters	[4] [5]
diag-banners	=*	Display POST banners	
save	=&	Persist Settings in EEPROM	
	=#	Set serial number	[4] [5]
move	@	Move current column cursor	
font	A	Select character font	
sound	B	Beep / Play sounds	
clear	C	Clear matrix display	
dim	D	Set light dimmer levels	
graph	H	Add histogram/bargraph data point	
bitmap	I	Draw bitmap graphic image	
color	K	Set current color	
morse	M	Send message via Morse code	
query	Q	Query full device status	[2] [5]
text	T	Display text on matrix	
alpha	V	Manipulate segmented LED display	[6]
current		Report current status display	
post		Add queued display message	
postlist		Report queued display message(s)	
unpost		Remove queued display message(s)	
update		Update user-defined variables	

[1] Busylight compatible command

[2] Sends response

[3] Busylight extension (not in original Busylight)

[4] USB only

[5] Must be targeted to a specific device at a time; does not respond on the global address

[6] On supported hardware only

Table 2.2: Summary of All Commands

Code	Hex	Description
~@	00	Never allowed in strings (null byte)
~C<pos>	03<pos>	Move current column cursor to <pos>
~D	04	Never allowed in strings (command terminator)
~F<digit>	06<digit>	Switch current font
~H<pos>	08<pos>	Move cursor left <pos> columns
~J<pos>	0A<pos>	Move cursor down <pos> rows
~K<rgb>	0B<rgb>	Change color to <rgb>
~L<pos>	0C<pos>	Move cursor right <pos> columns
~U<pos>	15<pos>	Move cursor up <pos> rows
~X<hex>	18<hex>	Specify character code as 2-digit hex value
~[	1B	Never allowed in strings (string terminator)

Table 2.3: Control Codes in String Values

scrolled through. **Not yet implemented. May be limitations on how many simultaneous transition effects may be playing at once, including scrolling text.**

Displays the text <string> by scrolling it across the display from right to left. If <loop> is “.”, the text is only scrolled once; if it is “L” then it repeatedly scrolls across the screen in an endless loop.

The text is rendered in the current font and may contain any 8-bit bytes except as otherwise noted (but avoiding ASCII control codes is wise to be safe from conflict with future control codes which may be added to the protocol). The string is terminated by an escape character (hex byte 1B₁₆), indicated in the protocol description as ESC.

The string may include control codes as listed in Table 2.3. These allow for special effects such as color and font changes to appear mid-message. They also include the three-character sequence ~X<hh> to add any arbitrary 8-bit codepoint specified as the 2-digit hex value <hh> to compensate for interference from your terminal program or transport protocols which may, e.g., encode non-7-bit-ASCII characters to a multi-byte UTF-8 representation which the readerboard doesn’t grok.

==Set Operational Parameters

0	1	2	3	4
=	<ad’>	<uspd’>	<rspd’>	<ad’ _G >

```

/readerboard/v1/configure-device?a=<ad>&rspeed=<baud>
&uspeed=<baud>&address=<ad’>&global=<ad’G>

```

This command sets a few operational parameters for the sign. Once set, these will be persistent across power cycles and reboots.

Code	Speed
0	300
1	600
2	1,200
3	2,400
4	4,800
5	9,600 (default)
6	14,400
7	19,200
8	28,800
9	31,250
A	38,400
B	57,600
C	115,200

Table 2.4: Baud Rate Codes

If the $\langle ad' \rangle$ parameter is “.” then the RS-485 interface is disabled entirely. Otherwise it is a value from 0–63 encoded as described in Table 2.5. This enables the RS-485 interface and assigns this sign’s address to $\langle ad' \rangle$. (In the HTTP API, use “address=.” to indicate this.)

The baud rate for the USB and RS-485 interfaces is set by the $\langle uspd \rangle$ and $\langle rspd \rangle$ values respectively. Each is encoded as per Table 2.4.

The $\langle ad'_G \rangle$ value is an address in the range 0–15 which is not assigned to any other device on the RS-485 network. This is used to signal that all readerboards should pay attention to the start of the command because it might target them either as part of a list of specific readerboards or because the command is intended for all readerboards at once. This is encoded in the same way as $\langle ad' \rangle$.

This command may only be sent over the USB port.

By default, an unconfigured readerboard is set to 9,600 baud with the RS-485 port disabled, but these defaults may be changed when compiling the firmware for specific devices.

=#—Set Serial Number

0	1	2	...			
=	#	$\langle Serial \rangle$		\$	#	=

To avoid recompiling the firmware image for every single device, the serial number may be set in the field using this command. The serial number is stored in EEPROM, so it won’t be persistent on units that are not equipped with EEPROM.

This command may only be sent over the USB port.

The serial number may be arbitrary alphanumeric text of up to six characters. Numbers RB0000–RB0299 and B00000–B00299 are reserved for use by the author.

=*—Display POST Banners

0	1	2
=	*	=

/readerboard/v1/diag-banners?a=<ad>

Causes the readerboard to display the same banners as it does during power-on self-test. Busylights don't display the equivalent series of messages when they power up, but if you send them this command they will send the same information as the readerboard POST banners, but will do so by flashing out the information in Morse code on the bottom LED.

=&—Persist Settings in EEPROM

0	1	2	3
=	&	<type>	=

/readerboard/v1/save?a=<ad>&type=<type>

Saves current system state in EEPROM so they remain in effect across reboots. Currently, the only supported <type> is “D” which saves the current dimmer settings.

?—Query Discrete LED Status

0
?

/readerboard/v1/busy?a=<ad>

This command causes the sign to send a status report back to the host to indicate what the discrete LEDs (usually showing a person's “busy” status) are currently showing.

This cannot be sent to the global device address, although you can request the server to query the global address in which case it will individually poll each device.

This response has the form:

0	1	2	3	4	5	6	7	8
L	0	$\langle led_0 \rangle$	$\langle led_1 \rangle$	$\langle led_2 \rangle$	$\langle led_3 \rangle$	$\langle led_4 \rangle$	...	$\langle led_{n-1} \rangle$
\$	F	flasher status (see below)						\$
S	strober status (see below)						\$	\n

The initial bytes “L0” identify the format of the data that follows. If the format changes in a way that breaks the previous parsing rules, the “0” will be incremented according to the conventions of 6-bit integer encoding as described in Table 2.5.

Each $\langle led_x \rangle$ value is a single character which is “_” if the corresponding LED is off, or the LED’s color code or position number if it is on. One such value is sent for each LED installed in the sign (typically eight for reader-boards), followed by a “\$” to mark the end of the list.

The flasher and strober status values are variable-width fields which indicate the state of the flasher (see F command) and strober (see * command) functions. In each case, if there is no defined sequence, the status field will be:

0	1
$\langle run \rangle$	_

or

0	1	2	3	4	5	6
/	$\langle up \rangle$	$\langle on \rangle$	$\langle down \rangle$	$\langle off \rangle$	$\langle run \rangle$	_

Otherwise, the state of the flasher or strober unit is indicated by:

0	1	2	3	4	...	$n+3$
$\langle run \rangle$	$\langle pos \rangle$	@	$\langle led_0 \rangle$	$\langle led_1 \rangle$	...	$\langle led_{n-1} \rangle$

or

0	1	2	3	4	5	6	7
/	$\langle up \rangle$	$\langle on \rangle$	$\langle down \rangle$	$\langle off \rangle$	$\langle run \rangle$	$\langle pos \rangle$	@
$\langle led_0 \rangle$	$\langle led_1 \rangle$	...	$\langle led_{n-1} \rangle$				

If the flasher has custom timings defined for it, a five-character block starting with “/” will appear at the start of the status and will contain the four timing values in the same format as specified to the F command when they were set (q.v.).

In any case, $\langle run \rangle$ is the ASCII character “S” if the unit is stopped or “R” if it is currently running. If there is a defined sequence, $\langle pos \rangle$ indicates the

7	6	5	4	3	2	1	0
0	0	1	1	$\langle n \rangle \quad (n < 16)$			
0	1	$\langle n \rangle - 16 \quad (n \geq 16)$					

Value	Code	Value	Code
0–9	0–9	17–42	A–Z
10	:	43	[
11	;	44	\
12	<	45	]
13	=	46	^
14	>	47	_
15	?	48	`
16	@	49–63	a–o

(Each code is the numeric value plus 48.)

Table 2.5: ASCII Encoded Integer Values (0–63)

0-origin position within the sequence of the light currently being flashed or strobed, encoded as described in Table 2.5. The $\langle led_x \rangle$ values are as allowed for the F or * command that set the sequence. (Regardless of the actual F or * command parameters, the report will show symbolic color codes where possible, or numeric position codes otherwise.)

The status message sent to the host is terminated by a newline character (hex byte 0A), indicated in the protocol description above as “\n”.

@—Set Column Cursor Position

0	1
@	$\langle pos \rangle$

/readerboard/v1/move?a= $\langle ad \rangle$ &pos= $\langle pos \rangle$

Sets the column cursor position to the value indicated by $\langle pos \rangle$. See Table 2.5.

A—Select Font

0	1
A	$\langle digit \rangle$

/readerboard/v1/font?a= $\langle ad \rangle$ &idx= $\langle digit \rangle$

Sets the font to use for rendering text with the < and T commands. The font codes for $\langle digit \rangle$ are listed in Table 2.6. The full text fonts support the printable ASCII characters plus a majority of the Unicode glyphs with codepoints less than 256. See Tables 2.7–2.9 for a complete font glyph listing with codepoint assignments.

Code	Font Description
0	Fixed-width 5×7 matrix plus descenders in 8th row
1	Variable-width version of font 0
2	Bold alphanumerics and special symbols

Table 2.6: Font Codes

Font 3 is a double-height font which is really only useful on boards with 16 or more rows of LEDs.

B—Beep/Play Sound Sequence

0	1	2	3	4	...
B	<i>⟨loop⟩</i>	<i>⟨note⟩</i>	<i>⟨cs (hex)⟩</i>	...	\$

/readerboard/v1/sound?a=⟨ad⟩&loop=⟨bool⟩¬es=⟨note⟩⟨cs⟩...

This command causes the device to play a sequence of notes on the speaker. If *⟨loop⟩* is “L”, the note sequence is played repeatedly until another B command is received. If the sequence is empty, all sound is stopped. If *⟨loop⟩* is “.”, it is only played once.

The sequence consists of a series of *⟨note⟩* values, each of which is followed by the note duration *⟨cs⟩* which is a two-character hexadecimal integer in centiseconds. (1/100th of a second). Each *⟨note⟩* begins with a letter A–G, optionally followed by # or b to indicate a sharp (#) or flat (b) note, respectively, followed by a digit 0–8 to indicate the octave. The notes range from B₀ (B₀ at 31 Hz) to D#₈ (D₈[#] at 4.978 kHz). A rest is indicated by “R” as the note letter.

An example 3-note attention tone where the first two notes are played for ½ second and the final one is held for a full second is “G432E532C564” (note G₄, 32₁₆ cs=0.50 s, note E₅, 32₁₆ cs=0.50 s, note C₅, 64₁₆ cs=1.00 s). Duration codes range from 00 (0₁₆=0₁₀ cs=0.00 s) to FF (FF₁₆=255₁₀ cs=2.55 s), in increments of 0.01 s.

The letters may be upper- or lower-case (the letter b is distinguished as a note name vs. a “b” half-step shift indicator from context).

Some devices are not able to play notes but can sound a beep at a fixed frequency (typically 490 Hz or just ever so slightly flat of B₄ for SparkFun Pro Micro and Arduino Mega 2560 microcontrollers and 1 kHz or between B₅ and C₆ for Arduino Due). These will play that single tone regardless of the note specified in the sequence.

	0	1	2	3	4	5	6	7	
00x	null			move	end		font		0x
01x	back		down	color	forw				
02x						up			1x
03x	hex			esc					
04x		!	"	#	\$	%	&	'	2x
05x	(	)	*	+	,	-	.	/	
06x	0	1	2	3	4	5	6	7	3x
07x	8	9	:	;	<	=	>	?	
10x	@	A	B	C	D	E	F	G	4x
11x	H	I	J	K	L	M	N	O	
12x	P	Q	R	S	T	U	V	W	5x
13x	X	Y	Z	[	\	]	^	_	
14x	'	a	b	c	d	e	f	g	6x
15x	h	i	j	k	l	m	n	o	
16x	p	q	r	s	t	u	v	w	7x
17x	x	y	z	{		}	~	///	
20x	“	”	‘	’	†	‡	...	'	8x
21x	”	!!	—	←	→	↑	↓	≠	
22x	≤	≥	≈	Γ	Δ	Ξ	Π	Σ	9x
23x	Ω	π	ρ	σ	—	‰			
24x		ı	¢	£	¤	¥	¦	§	Ax
25x	“	©	ª	«	¬	-	®		
26x	°	±	²	³		μ	¶	•	Bx
27x	‘	¹	º	»	¼	½	¾	¿	
30x	À	Á	Â	Ã	Ä	Å	Æ	Ç	Cx
31x	È	É	Ê	Ë	Ì	Í	Î	Ï	
32x	Ð	Ñ	Ò	Ó	Ô	Õ	Ö	×	Dx
33x	Ø	Ù	Ú	Û	Ü	Ý	Þ	ß	
34x	à	á	â	ã	ä	å	æ	ç	Ex
35x	è	é	ê	ë	ì	í	î	ï	
36x	ð	ñ	ò	ó	ô	õ	ö	÷	Fx
37x	ø	ù	ú	û	ü	ý	þ	ÿ	
	8	9	A	B	C	D	E	F	

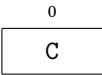
Table 2.7: Font Table for Fonts #0 and #1

	0	1	2	3	4	5	6	7	
00x	null			move	end		font		0x
01x	back		down	color	forw				
02x						up			1x
03x	hex			esc					
04x		!	□	■					2x
05x					,		.	thin .	
06x	0	1	2	3	4	5	6	7	3x
07x	8	9	:						
10x	©	A	B	C	D	E	F	G	4x
11x	H	I	J	K	L	M	N	O	
12x	P	Q	R	S	T	U	V	W	5x
13x	X	Y	Z						
14x	®	↶	↷	↲	↴	↵	↶	↷	6x
15x	↘	←	→	↑	↓	⬛	⬛		
16x	✓	✓	∞	Œ	œ	€	∴	∴	7x
17x	✕	◀	▶	▲	▼	↕	◆	◇	
20x	λ	Θ	Φ	Ψ	♂	♀		zero	8x
21x	±	≡	⊕	⊙	≡	clover	notes	EE	
22x	AM	PM	°F	°C	wide Œ	wide œ			9x
23x	The phases of the moon								
24x		TS1	TS2	TS3	TS4	TS5	TS6	TS7	Ax
25x	TS8								
26x	The eight trigrams 0–7								Bx
27x	8	9	A	B	C	D	E	F	

TS<n> = Thin space of <n> pixels

Table 2.8: Font Table for Font #2

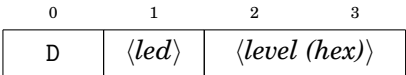
C—Clear Matrix Display



/readerboard/v1/clear?a=<ad>

Clears the matrix display so that no LEDs are illuminated. Does not affect the discrete LEDs.

D—Set Dimmer Levels



/readerboard/v1/dim?a=<ad>&l=<led>&d=<level>

	0	1	2	3	4	5	6	7	
00x	null			move	end		font		0x
01x	back		down	color	forw				
02x						up			1x
03x	hex			esc					
04x					\$	%			2x
05x	(	)	*	+	,	-	.	/	
06x	0	1	2	3	4	5	6	7	3x
07x	8	9	:	;	<	=	>		
10x									4x
11x									
12x									5x
13x				[		]	^	_	
14x									6x
15x									
16x									7x
17x									
20x									8x
21x									
22x									9x
23x									
24x									Ax
25x									
26x									Bx
27x					¼	½	¾		
	x0	x1	x2	x3	x4	x5	x6	x7	
	8	9	A	B	C	D	E	F	

Table 2.9: Font Table for Font #3

This command adjusts the apparent brightness of the LEDs. The brightness level is expressed as a two-digit hex value from 00₁₆ (fully off) to FF₁₆ (fully on). (In the HTTP API, as integers 0–255.)

The specific lamp to adjust is given by *<led>*, and can have any of the values shown in Table 2.10 to dim one of the discrete status LEDs, or “_” to dim the matrix LEDs, or “*” to dim all the discrete status LEDs to the same value.

Not all LEDs are dimmable, depending on the specific hardware capabilities. All LEDs on readerboard models from revision 3.x and after. On the busylight hardware, only the third, fifth, and seventh, LED tiers (usually assigned amber, red, and white colors) are dimmable.

Note that the readerboards were designed to run best at full brightness. Dimming the matrix display too much may result in the display appearing to flicker too much. This effect is most pronounced with white content, a

Code*	Light	Color
W	L ₇	white
B	L ₆	blue
b	L ₅	blue
R	L ₄	red
r	L ₃	red
Y	L ₂	amber
y	L ₁	amber
G	L ₀	green
-	—	(no LED/off)
0–9	L ₀ –L ₉	LED installed at physical position 0–9

*If a sign is built with different colors in these positions, the letter codes for those LEDs will match the custom color arrangement for that sign.

(Custom firmware modification required.)

Table 2.10: Discrete LED Codes and Colors

little less so with yellow, cyan, and magenta.

F—Flash Lights in Sequence

	0	1	2	3	...	
F	$\langle led_0 \rangle$	$\langle led_1 \rangle$	$\langle led_2 \rangle$	$\dots$	$\langle led_{n-1} \rangle$	\$

or

	0	1	2	3	4	5	6	...	
F	/	$\langle up \rangle$	$\langle on \rangle$	$\langle down \rangle$	$\langle off \rangle$	$\langle led_0 \rangle$	$\dots$		\$

`/readerboard/v1/flash?a= $\langle ad \rangle$ &l= $\langle led_0 \rangle \langle led_1 \rangle \langle led_2 \rangle \dots \langle led_{n-1} \rangle$
&up= $\langle sec \rangle$ &on= $\langle sec \rangle$ &down= $\langle sec \rangle$ &off= $\langle sec \rangle$`

Each $\langle led \rangle$ value is an ASCII character corresponding to a discrete LED as shown in Table 2.10. Note that the assignment of colors to these LEDs is dependent on your particular hardware being assembled that way. As an open source project, of course, you (or whomever assembled the unit) may choose any color scheme you like when building the board.

An $\langle led \rangle$ value of “-” means there is to be no LED illuminated at the corresponding position in the sequence.

Up to 64 $\langle led \rangle$ codes may be listed. The sign will cycle through the sequence, lighting each specified LED briefly before moving on to the next one. The sequence is repeated forever in a loop until an L, S or X command is received.

If only one $\langle led \rangle$ is specified, that light will be flashed on and off. Setting an empty sequence (no codes at all) stops the flasher's operation.

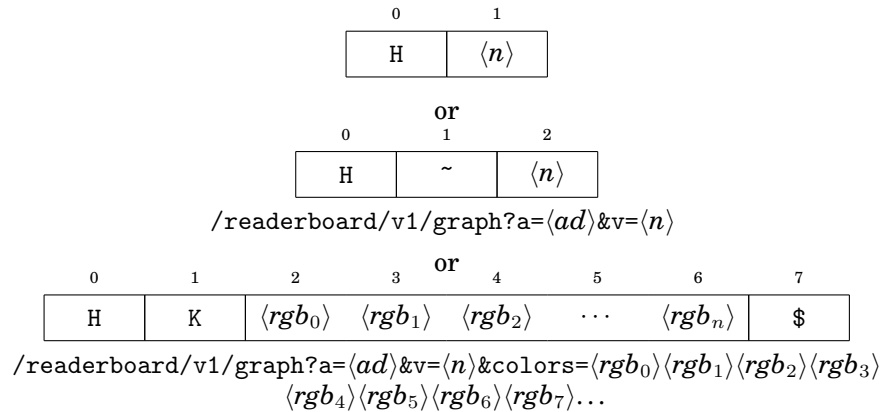
The sequence is terminated by either a dollar-sign (“\$”) character or the escape control character (hex byte $1B_{16}$), indicated in the protocol diagram above simply as “\$”.

Optionally, a block of timing values may be given before the block of $\langle led \rangle$ values. This is introduced by a “/” character and consists of four 6-bit encoded integer values giving the time interval over which the light should fade up to full brightness, the time to remain on, the interval over which to dim down, and the time to remain off. These values are in units of $\frac{1}{10}$ th of a second (thus ranging from 0–6.4 seconds). For LEDs which do not support dimming, the values of $\langle up \rangle$ and $\langle down \rangle$ are treated as if they were zero, instantly turning on or off the LED. (In the HTTP API, the $\langle sec \rangle$ values are floating-point numbers of seconds; e.g., a value of 1.2 seconds would be expressed in the API as “up=1.2” and in the hardware protocol as “<”).

If you have set a dimmer level for any of the LEDs, then the maximum brightness that they will ramp up to and down from will be that dimmer level. If the dimmer is set very low, then the effect of $\langle up \rangle$ and $\langle down \rangle$ won't be too noticeable since you didn't leave it much room to dim up and down within that range.

This command may be given in upper- or lower-case (“f” or “F”).

H—Draw Bar Graph Data Point



This command is used to draw a bar-graph element. Repeating this command causes a scrolling data display which shows a set of data samples over some period of time.

In the simplest form, it takes a single byte parameter. The value $\langle n \rangle$ is an integer encoded as per Table 2.5, and is drawn in the far-right column of the matrix display, as a column of $\langle n \rangle$ lights stacked up from the bottom row (a value of 0 results in no lights). All existing matrix data are scrolled left one column.

In earlier versions of the readerboard hardware, devices with 27 or more rows weren't supported. Now that they are, the value “K” makes this command ambiguous since HK could be the start of a bar graph command with explicit color codes, or a single point with value 27. To resolve the ambiguity, an optional “~” character may be sent after the initial H to explicitly indicate that the following byte is a data value. Thus, HK is always the start of an explicitly-colored bar, and H~K is how you would send a data bar of 27 pixels in height. You **may** start all data points with H~ but you only **must** do so if the value encodes to K.

In the second form, it takes a number of $\langle rgb \rangle$ values (see Table 2.12) which give the color to light up each row in the bar-graph column. Unlike other drawing commands, but to make the most logical sense for histograms, $\langle rgb_0 \rangle$ refers to the bottom-most LED in the column, so the number of color values in the list is the number of data points being graphed.

In HTTP, the value of $\langle n \rangle$ is sent as an integer value.

I—Draw Bitmap Image

0	1	2	3	4	5	6	7
I	$\langle merge \rangle$	$\langle pos \rangle$	$\langle trans \rangle$	$\langle height \rangle$	$\langle R coldata \rangle \dots$		\$
$\langle G coldata \rangle \dots$		\$	$\langle B coldata \rangle \dots$		\$	$\langle flashdata \rangle \dots$	
\$							

```
/readerboard/v1/bitmap?a=<ad>&merge=<bool>&pos=<pos>
&trans=<trans>&image=<R coldata>...$<G coldata>...
$<B coldata>...$<flashdata>&height=<height>
```

Draws an arbitrary bitmap image onto the matrix display starting at column $\langle pos \rangle$, encoded as per Table 2.5. A $\langle pos \rangle$ value of “~” represents the current column cursor position.

Each column data, from left to right, are given by $\langle coldata \rangle$ values, each of which is a two-digit ASCII hexadecimal value for each byte of height (the number of bytes of height are given by the $\langle height \rangle$ value), with the least-significant bit of the first such digit pair representing the top row of the matrix.

The column data values are terminated by either a dollar-sign (“\$”) character or the escape control character (hex byte 1B₁₆), indicated in the protocol diagram above simply as “\$”.

They may be short (or entirely empty if the corresponding color plane has no pixels set).

The first set of column data provide the bits for the red color plane. After the terminating byte (\$ or ESC), two more sets of column data are sent, each with identical format to the first, to provide the bits for the green and blue color planes, respectively. The final plane, $\langle flashdata \rangle$, causes the corresponding pixel to flash for any bits set to 1. Monochrome boards

Code	Transition
.	No transition
>	Scroll in from left
<	Scroll in from right
^	Scroll up from bottom
v	Scroll down from top
L	wipe left
R	wipe right
U	wipe up
D	wipe down
	wipe left and right from middle column
-	wipe up and down from middle row
?	choose a random transition

Table 2.11: Transition Effect Codes

accept only two color bitplanes: the bitmap of pixels to light and then the flash plane.

The column cursor is moved to be after the end of the image.

If $\langle merge \rangle$ is the character “.” then each column’s contents is cleared before setting the pixels as per the $\langle coldata \rangle$ value. If $\langle merge \rangle$ is “M” the bits set in $\langle coldata \rangle$ are added to the lit pixels already in the column.

The $\langle trans \rangle$ value indicates the transition effect to use when adding the image to the display. See Table 2.11.

K—Set Current Color

0	1
K	$\langle rgb \rangle$

/readerboard/v1/color?a= $\langle ad \rangle$ &color= $\langle rgb \rangle$

This command sets the color which all future commands will use by default (although some of them allow for the specification of colors directly on an *ad hoc* basis, which does not affect the current default color).

The $\langle rgb \rangle$ value is a value encoded as defined in Table 2.12. Note that anything drawn in black simply turns off the corresponding pixels.

L—Light Multiple LEDs

0	1	2	3	...	n	$n + 1$
L	$\langle led_0 \rangle$	$\langle led_1 \rangle$	$\langle led_2 \rangle$	...	$\langle led_{n-1} \rangle$	\$

/readerboard/v1/light?a= $\langle ad \rangle$ &l= $\langle led_0 \rangle \langle led_1 \rangle \langle led_2 \rangle \dots \langle led_{n-1} \rangle$

7	6	5	4	3	2	1	0
0	0	1	1	<i>flash</i>	<i>blue</i>	<i>green</i>	<i>red</i>

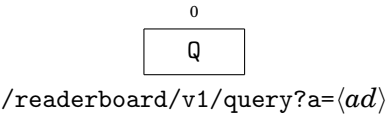
Code	Color	Code	Color
0	off	8	off
1	red	9	flashing red
2	green	:	flashing green
3	amber	;	flashing amber
4	blue	<	flashing blue
5	magenta	=	flashing magenta
6	cyan	>	flashing cyan
7	white	?	flashing white

Table 2.12: Color codes for *rgb* parameters

This command is identical to the S command (see below), except that multiple discrete LEDs can be specified, all of which are illuminated simultaneously. See Table 2.10. Note that if a strobe sequence is running (via a previous * command), it remains running.

The list of *led* values is terminated by either a dollar sign (\$) character or an escape byte (hex value 1B), represented in the protocol diagram as “\$”.

Q—Query Readerboard Status



This command causes the sign to send a status report back to the host to indicate the general status of the device, including the discrete LED display which may be queried using the ? command.

This cannot be sent to the global device address, although you can request the server to query the global address in which case it will individually poll each device.

The response has the form:

0	1	2	3	4	5	6	7	8
Q	0	$\langle model \rangle$	=	$\langle ad \rangle$	$\langle uspd \rangle$	$\langle rspd \rangle$	$\langle ad_G \rangle$	$\langle EE \rangle$
$\langle spkr \rangle$	\$	V	$\langle hwversion \rangle$	\$	R	$\langle romversion \rangle$		
\$	S	$\langle serial \rangle$	\$	L	$\langle led_0 \rangle$	$\langle led_1 \rangle$	$\langle led_2 \rangle$	$\langle led_3 \rangle$
$\langle led_4 \rangle$	...	$\langle led_{n-1} \rangle$	\$	F	flasher status (q.v.)			
\$	S	strober status (q.v.)					\$	D
$\langle dim_m \rangle$		$\langle dim_0 \rangle$	...	$\langle dim_n \rangle$	\$	M		
$\langle width \rangle$		$\langle height \rangle$		$\langle R coldata \rangle$	...	\$		
$\langle G coldata \rangle$	...	\$	$\langle B coldata \rangle$	...	\$			
$\langle flashdata \rangle$	...	\$	$\langle alen \rangle$	$\langle adata \rangle$	...	\$		\n

The leading “Q0” indicates the format of the data that follows. If a change is introduced that breaks the parsing rules needed to understand the response, the “0” will be incremented as documented in Table 2.5 for 6-bit integer encoding.

The $\langle model \rangle$ field may be “B” for the stand-alone busylight modules which only include discrete busy status lights, “M” for $\langle width \rangle \times \langle height \rangle$ matrix display hardware with monochrome LEDs, “C” for $\langle width \rangle \times \langle height \rangle$ RGB color display boards, or “Q” for the “QS” variant of color display boards with expanded discrete LED support and alphanumeric segmented displays. Note that $\langle width \rangle$ and $\langle height \rangle$ are encoded 12-bit integers, where the most significant 6 bits are sent first in each pair and the full encoded value is one less than the actual width or height. Thus, dimensions of 1–4,096 pixels in each direction may be reported.

$\langle hwversion \rangle$ and $\langle romversion \rangle$ indicate the versions, respectively, of the hardware the firmware was compiled to drive, and of the firmware itself. Each of these fields are variable-width and conform to the semantic version standard 2.0.0.⁴ Each is terminated by a dollar-sign (\$) character (and thus those strings may not contain dollar signs).

The $\langle serial \rangle$ field is a variable-width alphanumeric string which was set when the firmware was compiled. It should be a unique serial number for the device (although that depends on some effort on the part of the person compiling the firmware to insert that serial number each time). Serial numbers RB0000–RB0299 are reserved for the original author’s use. This string is also terminated with a dollar sign.

The $\langle EE \rangle$ field is “X” if the device is fitted with an external EEPROM chip or module, “I” if using the EEPROM memory on the Arduino controller board, or “_” if there is no EEPROM memory at all (and thus, configuration settings made with the = commaand are not persistent).

⁴See semver.org.

The $\langle spkr \rangle$ field is “T” if the hardware supports a speaker with full tone output, “S” if the speaker can only emit a single fixed tone, or “_” if no audio output is supported.

The dimmer values appear in a block beginning with “D”. Each dimmer value is a two-digit hex value. $\langle dim_m \rangle$ is the dimmer value for the LED matrix while $\langle dim_n \rangle$ is the dimmer level for discrete status LED # n . The block is terminated with “\$”. These values are given as “_” if the device does not have the corresponding feature (e.g., matrix) or the corresponding LED is not dimmable on that device.

The discrete light, flasher, and strober fields (starting, respectively, with L, F, and S, are as documented above for the ? (busy) command. Busylight units stop here and don’t include the image bitmap data that readerboards report.

The $\langle coldata \rangle$ bytes are sent just as with the I command, as hexadecimal values indicating the LEDs lit on the matrix display, with $\langle coldata_0 \rangle$ being the leftmost column of the display and $\langle coldata_{63} \rangle$ being the rightmost. In each column, the least significant bit indicates the LED on the top row.

For devices which have 14-segment alphanumeric LED displays, the number of supported digits is sent as a 6-bit integer value $\langle alen \rangle$ (or 0 if there are no such displays on this device). This is followed by a number of four-digit big-endian hex words, one per LED digit, which indicate the currently-lit segments on the display. In each word, the bits correspond to segments a, b, c, . . . , l, m, n, and the decimal point, from bit 0 to bit 14. Bit 15, the MSB, is always clear.

Note that monochrome boards only send a single color plane of data plus the flash plane.

The $\langle ad \rangle$, $\langle ad_G \rangle$, $\langle uspd \rangle$, and $\langle rspd \rangle$ values are as last set by the = command (or the factory defaults if they were never changed).

The status message sent to the host is terminated by a newline character (hex byte $0A_{16}$), indicated in the protocol description above as “\n”.

For Busylight devices, serial numbers B0000–B0299 are reserved for use by the original author.

M—Morse Code

0	1	2	...
M	$\langle led \rangle$	$\langle message \rangle$	ESC

`/readerboard/v1/morse?a= $\langle ad \rangle$ &t= $\langle message \rangle$ &l= $\langle led \rangle$`

Sends $\langle message \rangle$ by blinking the designated status LED in Morse code. If $\langle led \rangle$ is “_”, then the message is sent audibly over the speaker.

Supported characters include the letters A–Z, digits 0–9, and the symbols !, ", \$, &, ', (,), +, ,, -, ., /, :, ;, ?, @, and _ . At the end of the message, the prosign $\overline{SK}$ is transmitted (which is also available at codepoint 3).

Supported prosigns include:

Sign	Meaning	Codepoint	
$\overline{\text{AR}}$	start message	2 ₁₀	02 ₁₆
$\overline{\text{AS}}$	wait	19 ₁₀	13 ₁₆
$\overline{\text{BT}}$	break	30 ₁₀	1E ₁₆
$\overline{\text{DDD}}$	relayed distress	18 ₁₀	12 ₁₆
$\overline{\text{HH}}$	correction	127 ₁₀	7F ₁₆
$\overline{\text{KA/CT}}$	attention	7 ₁₀	07 ₁₆
$\overline{\text{SK}}$	end of message	3 ₁₀	03 ₁₆
$\overline{\text{SOS}}$	distress	17 ₁₀	11 ₁₆
$\overline{\text{VE/SN}}$	verified	6 ₁₀	06 ₁₆

This is intended for diagnostic messages or signalling when other, easier to read, methods aren’t available. As such, other device operations may be suspended while the message is being sent by Morse code.

S—Light Single LED

0	1
S	$\langle led \rangle$

/readerboard/v1/light?a= $\langle ad \rangle$ &l= $\langle led \rangle$

Stops the flasher (cancelling any previous F command) and turns off all discrete LEDs. The single LED indicated by $\langle led \rangle$ is turned on. See Table 2.10. Note that if a strobe sequence is running (via a previous * command), it remains running.

This command may be given in upper- or lower-case (“s” or “S”).

T—Display Text

0	1	2	3	4	...	n+4
T	$\langle merge \rangle$	$\langle align \rangle$	$\langle trans \rangle$	$\langle string \rangle$		ESC

or

0	1	2	3	4	5	6	7	n+7
T	@	$\langle row \rangle$	$\langle height \rangle$	$\langle merge \rangle$	$\langle align \rangle$	$\langle trans \rangle$	$\langle string \rangle$	ESC

/readerboard/v1/text?a= $\langle ad \rangle$ &merge= $\langle bool \rangle$
&align= $\langle align \rangle$ &trans= $\langle trans \rangle$ &t= $\langle string \rangle$
&row= $\langle row \rangle$ &height= $\langle height \rangle$

Displays the text $\langle string \rangle$ at the current cursor position. The cursor position is then moved past the text to be ready for the next string to be printed.

Code	Alignment Behavior
.	No alignment (print at cursor)
<	Left (print at left edge of matrix)
>	Right (end of text at right edge of matrix)
~	Center (between left and right edges)
R	Local right (end of text at cursor)
C	Local center right (between cursor and right edge)
L	Local center left (between left edge and cursor)

Table 2.13: Alignment Codes

In the second form, the `<row>` and `<height>` parameters define a region of the display that extends to the full width of the display but includes `<height>` pixel rows starting at pixel #`<row>` (with row #0 being the topmost row of the device). In this case, the command behaves as though this region is the only part of the sign that exists as far as where characters will be drawn, transition effects performed, previous data cleared, etc. **Not yet implemented. May be limitations on how many simultaneous transition effects may be playing at once, including scrolling text.**

The `<align>` code indicates how the text is aligned on the display using one of the codes shown in Table 2.13.

If `<merge>` is “M” then the text is drawn without disturbing any existing pixels around it, adding to what is already there. If it is “.” then everything to the right of the cursor is cleared before drawing the text. (Alternatively, this may clear everything to the left of the cursor or the entire screen, depending on the alignment mode in use.)

The `<trans>` code specifies the transition effect to be used to add this string to the display as shown in Table 2.11.

The text is rendered in the current font and may contain any 8-bit bytes except as otherwise noted (but avoiding ASCII control codes is wise to be safe from conflict with future control codes which may be added to the protocol). The string is terminated by an escape character (hex byte 1B), indicated in the protocol description as ESC. (Since a dollar sign may appear in `<string>`, the terminator must be an escape character for this command.)

The string may include control codes as listed in Table 2.3.

V—Manipulate Segmented LED Display

For units which include 14-segment alphanumeric character displays, this set of commands manipulates what is displayed there. There are five distinct subcommands which are grouped together here:

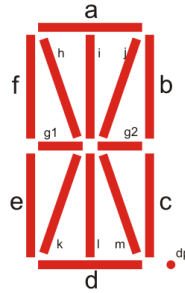


Image: Wikimedia Commons (public domain)

Figure 2.1: 14-Segment LED Alphanumeric Display

0	1	2	3	4	...	$n+4$
V	A	$\langle pos \rangle$	$\langle img \rangle$	$\langle text \rangle$		ESC

`/readerboard/v1/alpha?a= $\langle ad \rangle$ &pos= $\langle pos \rangle$ &img= $\langle bool \rangle$ &t= $\langle text \rangle$`

Starting with character position $\langle pos \rangle$ (with 0 being the leftmost character), write the characters of the $\langle text \rangle$ string into successive positions. Any character positions to the left of $\langle pos \rangle$ or to the right of the $\langle text \rangle$ are left unchanged.

If $\langle pos \rangle$ is the special code “#”, the entire display is replaced with $\langle text \rangle$, which is scrolled across the display once until it is fully visible, then the scrolling stops. If $\langle pos \rangle$ is “%”, the behavior is the same but the scrolling loops forever instead of stopping.

If $\langle img \rangle$ is “.”, then $\langle text \rangle$ is an ASCII string of characters which will be rendered using the device’s built-in character set. A period (“.”) character in $\langle text \rangle$ does not consume an additional character if it follows another character in the string; in this case the decimal point on the previous character’s segmented display will be lit.

However, if $\langle img \rangle$ is “I”, then $\langle text \rangle$ specifies the individual set of LED segments to light up for each affected character position. Each character to display is sent as a two-byte big-endian value expressed as a four-digit hex value that provides a bitmap of all fifteen LED segments that should be illuminated for that character.

Referring to the diagram in Figure 2.1, the LSB (bit 0) of each word is set (1) if segment “a” is lit, bit 1 for segment “b”, and so on, up to bit 14 for the decimal point. The MSB, bit 15, is always clear (0).

0	1	2	3
V	X	$\langle pos \rangle$	$\langle end \rangle$

/readerboard/v1/alpha-clear?a=⟨ad⟩&pos=⟨pos⟩&clearto=⟨end⟩

Turns off character positions starting with ⟨pos⟩, through ⟨end⟩, inclusive.

0	1
V	C

/readerboard/v1/alpha-clear?a=⟨ad⟩

Turns off all character positions.

X—Turn off Discrete LEDs

0
X

/readerboard/v1/off?a=⟨ad⟩

Turns off the flasher, strober, and all discrete LEDs.

This command may be given in upper- or lower-case (“x” or “X”).

SERVER-ONLY COMMANDS

The enjoyment of one's tools is
an essential ingredient of
successful work.

—Donald E. Knuth

FOR THE MOST PART the server exists as a way to host readerboards and busylights from a central machine while allowing users on other networked machines to post messages to them. It also provides a solution to the lack of an ability for signs to coordinate content with each other. If you use the server's API to tell a set of target signs to change their display contents, that will change whatever the sign was doing to what you are now telling it to do, even if other users had put their own messages on it.

This chapter describes server-side coordination between users in a network environment by providing a set of commands which clients can post to the server to request information to be displayed for them alongside other content other users have posted. It also provides some services to keep boards updated with information known to the server which the devices don't themselves track, such as time of day.

current—Report Current Display State

`/readerboard/v1/current?a=<ad>`

Returns a JSON-formatted report as an object with device IDs as the keys and values matching the result of the busy command, which gives the server's understanding of what should be displayed on the device(s). This

Code	Name	Code	Name
.	none	R	wipe-right
>	scroll-right	U	wipe-up
<	scroll-left	D	wipe-down
^	scroll-up		wipe-horiz
v	scroll-down	-	wipe-vert
L	wipe-left	?	random

Table 3.1: Transition codes for use in post command

Code	Name	Code	Name
.	none	C	local-center-right
>	right	L	local-center-left
<	left	R	local-right
^	center		

Table 3.2: Alignment codes for use in post command

differs from busy in that this does not go through the time and effort to poll each device to ask about the hardware status. It merely reports what the server last told the devices to display.

post—Post a Message to the Displays

```
/readerboard/v1/post?a=<ad>&t=<text>&id=<id>&trans=<trans>
&until=<datetime>&hold=<duration>&color=<rgb>&visible=<duration>
&show=<duration>&repeat=<duration>
```

Adds a text message to the display list. The message may include the special tokens listed in Table 3.4. The server will maintain the list of posted images and will update the devices as needed to display dynamic content.

If any lower-level display matrix commands are received that directly manipulate the matrix, the display of posted messages is suspended in favor of the directly set content. When a clear command is received, then display of the posted message list will resume.

The *<id>* value must be unique but the server doesn't assign these in order to keep the API simple. Thus, it is recommended that you assign these as UUIDs or other scheme to keep them from colliding.

If trans is given, this message will be introduced using the specified transition effect, specified by name or number (see Table 3.1).

If until is given, the message will automatically unpost when the specified date and time is reached. *<Datetime>* has one of the formats:

Code	Name	Code	Name
0	off	8	flashing-off*
1	red	9	flashing-red
2	green	10 or :	flashing-green
3	amber/yellow	11 or ;	flashing-amber/flashing-yellow
4	blue	12 or <	flashing-blue
5	magenta	13 or =	flashing-magenta
6	cyan	14 or >	flashing-cyan
7	white	15 or ?	flashing-white

*Included for technical completeness but actually using this code would be silly.

Table 3.3: Color codes for use with the post command

$$[\langle day \rangle @ [\langle hour \rangle] : \langle min \rangle [: \langle sec \rangle]$$

$$\langle day \rangle$$

$$\langle duration \rangle$$

In the first form, an absolute time is specified for the message’s expiration. In the second form, a day of the week is specified without a time (defaulting the time to 00:00:00), and in the third form, a relative duration since posting is given.

If $\langle day \rangle$ is specified, it gives the day of the week. $\langle Hour \rangle$ and $\langle sec \rangle$ default to the current hour and 00 respectively, so “until=monday” shows the message until the beginning of Monday; “until=tuesday@12:00” expires at noon on Tuesday, “until=:30” expires after half-past the current hour, “until=17:00” expires at 5:00 PM, and “until=30m” expires after 30 minutes have passed since posting.

If hold is given, the message will remain on-screen for the specified time duration before moving on to the next posted message.

If color is given, the current color will be changed to the specified value before writing the message; otherwise (or additionally) color codes may be added in-line in the message text. (See Table 3.3 for the codes you can use here.)

The show value gives a time duration during which the message will be visible on the sign. After this time has elapsed, it will be hidden (but not necessarily removed from the queue). If the repeat value is also specified, then the message will reappear after the specified duration has elapsed since it was hidden.

The visible parameter controls absolute time ranges outside of which the message will be unconditionally hidden. Hidden messages automatically reappear at the start of any of these visible time ranges. The value is a pair of date strings in any of the forms allowed for until, separated by a hyphen (‘-’). Multiple such pairs may appear, separated by commas (‘,’).

Code	Description
{big}	Display letters and numbers in large font
{color<n>}	Switch to color number <n> (as ^K<n>)
{<colorname>}	Switch to color by name
{date}	Date as “<dd>-<mmm>-<yyyy>”
{font<n>}	Switch to font number <n> (as ^F<n>)
{normal}	Stop displaying numbers in large font
{mdy}	Date as “<mm>/<dd>/<yyyy>”
{pom}	Phase of the moon (as a symbol)
{time12}	Time as “<hh>:<mm> AM/PM” on 12-hour clock*
{time24}	Time as “<hh>:<mm>” on 24-hour clock†
{ymd}	Date as “<yyyy>-<mm>-<dd>”
{@[±]<n>}	Reposition cursor to column <n>‡
{^[±]<n>}	Reposition cursor up [down] by <n> rows
{#<n>}	Insert character with codepoint <n> (decimal)
{#<h>}	Insert character with codepoint <h> (hex)
{\${name}}	User variable <name>
{ {	Literal “{”
} }	Literal “}”

*Hour is space-padded (e.g., “_3:15”)

†Hour is zero-padded (e.g., “03:15”)

‡-<n> and +<n> move left or right by <n> columns

Table 3.4: Special Codes in Post Messages

postlist—Report of Message Queue

/readerboard/v1/postlist?a=<ad>&id=<id>

Returns a JSON-formatted list of the messages in the display queue. If <ad> is missing or is the global address, the messages for all devices will be returned; otherwise the list is filtered to include only messages targetted at the listed devices.

If id is given, only that message is included in the report; if the <id> value starts with a slash (/) then the remaining characters are interpreted as a regular expression. Only messages whose <id> values match the regular expression are included in the report.

unpost—Remove a Posted Message

/readerboard/v1/unpost?a=<ad>&id=<id>

Remove the message with the given <id> from the display list. If <ad> is missing or is the global address, the message is removed for all devices. Otherwise, it is only removed for the listed target devices.

If the $\langle id \rangle$ value starts with a slash (‘/’) then the remaining characters are interpreted as a regular expression. All messages whose $\langle id \rangle$ values match the regular expression are unposted. Thus, “id=/.*” deletes the entire message queue.

update—**Update Dynamic Content**

`/readerboard/v1/update?<name0>=<value0>&<name1>=<value1>...`

For each $\langle name \rangle$ and $\langle value \rangle$ pair, define or update the corresponding user variable (usable in posted messages as $\{\$ \langle name \rangle\}$). The updated value will be displayed from this point until it is updated again.

PINOUTS

The nicest thing about
standards is that there are so
many of them to choose from.

—Andrew S. Tanenbaum

THIS CHAPTER DESCRIBES each connector used by the readerboard devices and what signals are present on which pins. Note that this begins with the current hardware revision, which is most likely the only part you need to know about. Following that are the connector descriptions for older, legacy versions of the hardware. Be careful to know which version you need to refer to.

8p8c RS-485 Connectors

There is no universal standard for RS-485 jack wiring, but often uses an 8p8c or 6p6c modular jack.¹

A few popular *de facto* standards are in use, but mostly everyone either uses discrete screw terminal blocks or makes up their own connector wiring arrangement.

¹Commonly, but not *entirely* correctly, referred to as “RJ-45” and “RJ-11” jacks. The “registered jack” standards RJ-45 and RJ-11 refer to the wiring arrangement of signals in a modular jack, not just the shape of the jack itself.

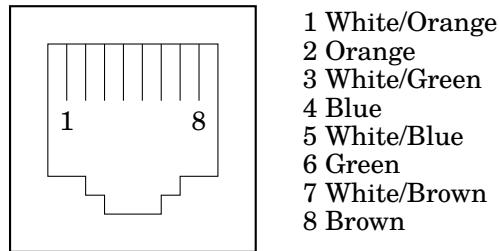


Figure 4.1: Female 8p8c Modular Jack with T-568B Color Codes

Lumos Wiring Pinout

The author's personal wiring standard used by all his RS-485-capable projects, starting with Lumos, supports full-duplex communication (i.e., a data pair that is dedicated to signals from the host PC out to all the connected units—but which could reverse direction if used with half-duplex devices—and a return data pair dedicated to signals from the networked devices back to the host) and a cable-check pair which can be used to power very small devices such as the Busylight or to verify that the entire set of daisy-chained devices are all fully connected and terminated.

The pinout is as follows:

Pin	Signal
1	Return data A (+)
2	Return data B (−)
3	Cable check send (outbound +9–16 V)
4	Data B (−)
5	Data A (+)
6	Cable check return
7	Power Ground
8	Signal Ground

DIN-8 Busylight Tree Connectors

The Busylight control boards use a DIN-8 jack where the light tree cable plugs in to carry the light-signal voltages out to the light tree itself. The pinout used is:

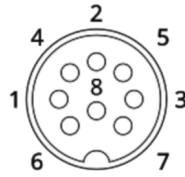


Image: adapted from original by Ancillalover, Wikimedia Commons, CC-BY-SA 4.0 int'l.

Figure 4.2: DIN-8 Jack (looking into socket from front)

Pin	Tier	Light Color	Wire Color
1	6	blue	blue
2	3	amber	white/brown
3	5	red	white/orange
4	4	red	orange
5	7	white	white/blue
6	2	amber	brown
7		+5 V power	green
8	1	green	white/green

Connectors for Version 3.4 Boards

Arduino Bus Connectors [J0]

Two parallel rows of stacking pins, for a total of 50 pins, extend out of the back of the readerboard PCB. An Arduino Mega 2560 or Due microcontroller board is connected to these pins.

The USB connector on the Arduino board may be used to directly connect the readerboard to a PC for initial configuration and/or to use it as a directly-connected singleton device. (If there are multiple devices in use, it may be preferable to switch to the RS-485 network after initial configuration of the device.)

Version 3.4.1 Boards

The external connections changed significantly at revision 3.3.1. Now J1 is a pair of push-in terminal blocks with a combined total of 16 wire connections to make it easy to connect a pair of CAT6 cables (cable “A” which carries the incoming data signals, and cable “B” which carries the same signals out to the next daisy-chained unit on the RS-485 serial network). The expanded terminal block eliminates the need to make off-board connections or splices between the cables.

The terminals are labeled as “A1” (cable A, pin 1), “B2” (cable B, pin 2) and so forth:

Pin	Wire	Signal	Pin	Wire	Signal
1	A1	Return Data A(+) In*	9	A7	Ground In
2	B1	Return Data A(+) Out*	10	B7	Ground Out
3	A2	Return Data B(-) In*	11	A8	Ground In
4	B2	Return Data B(-) Out*	12	B8	Ground Out
5	A3	Cable Check Send + In*	13	A5	Data A(+) In
6	B3	Cable Check Send + Out*	14	B5	Data A(+) Out
7	A6	Cable Check Return In*	15	A4	Data B(-) In
8	B6	Cable Check Return Out*	16	B4	Data B(-) Out

*Signal not used by the readerboard unit (passed through).

If this device is the last in the RS-485 network chain, insert a 120 Ω resistor between the A and B terminals that would have been used as the output if there had been another device connected there (e.g., B4 and B5). This properly terminates the RS-485 network at that point.

The power supply to the readerboard unit is on terminal block J5, with +9 V DC and ground connections as marked on the board.

J6 allows the connection of an external speaker, driven by a simple amplifier formed by C13, R54, and Q24.

Version 3.3.0 Boards

The previous revision, 3.3.0, used a single terminal block J1 to carry the power supply and the RS-485 signals we actually use, leaving the other wires of the serial cables to be connected externally.

The single external connector on version 3.3.0 boards is an 8-pin screw terminal block. This accepts a +9 V DC power input and ground on pins 8 and 7 respectively, which powers the entire board and the attached Arduino controller. If RS-485 communications will be used, the incoming signal is received on pins 1, 2, and 6 while the outgoing signal is on pins 3, 4, and 5. (In actuality, the “input” and “output” sense is arbitrary and either set of A and B signals may be used as input or output.)

If this device is the last in the RS-485 network chain, insert a 120 Ω resistor between the A and B terminals that would have been used as the output if there had been another device connected there. This properly terminates the RS-485 network at that point.

Pin	Signal	Pin	Signal
1	A (Data in +)	5	GND
2	B (Data in -)	6	GND
3	A (Data out +)	7	GND
4	B (Data out -)	8	+9 V DC in

The pinout of J1 for revision 3.2.2 boards is different:

Pin	Signal	Pin	Signal
1	A (Data in +)	5	GND
2	B (Data in -)	6	GND
3	+9 V DC in	7	A (Data out +)
4	GND	8	B (Data out -)

Adafruit Matrix Versions

There are two experimental versions of readerboard shields which drive the various matrix boards available from Adafruit.² These use a 16-pin ribbon cable which employs signalling that is similar in principle (although not identical) to our original design. Our compatible shield boards provide these signals on their J7 connector:

Pin	Signal	Pin	Signal
1	R ₁ red data	2	G ₁ green data
3	B ₁ blue data	4	Ground
5	R ₂ red data	6	G ₂ green data
7	B ₂ blue data	8	E (A ₄) row select or ground
9	A (A ₀) row select	10	B (A ₁) row select
11	C (A ₂) row select	12	D (A ₃) row select or ground
13	Clock	14	Latch
15	Output Enable	16	Ground

The standard adafruit-compatible shield supports eight busylight-like discrete LED outputs as well, but instead of them being soldered to the board as they would be on the original readerboard, ribbon cable connector J8 connects to external LEDs, which are assumed to have their own current-limiting resistors:

Pin	Signal	Pin	Signal
1	+9V common anode power	2	Ground
3	L7 cathode	4	L6 cathode
5	L5 cathode	6	L4 cathode
7	L3 cathode	8	L2 cathode
9	L1 cathode	10	L0 cathode

The remaining external connections, including J1 to RS-485 network, J5 to +9V power supply, and J6 to external speaker, are as for the original readerboard unit.

²And probably others, but that's just where the author got his from.

QSCC Variant

A slight variation of the Adafruit-compatible shields, the so-called “QSCC” model, is used for game applications in another project by the author, but may be useful in a variety of other situations as well. This is the same as what is listed above, except it does not offer a speaker output, and adds additional discrete LED outputs on J8 with the following ribbon cable pinout:

Pin	Signal	Pin	Signal
1	L0 cathode	2	L1 cathode
3	L2 cathode	4	L3 cathode
5	L4 cathode	6	L5 cathode
7	L6 cathode	8	L7 cathode
9	L8 cathode	10	L9 cathode
11	L10 cathode	12	L11 cathode
13	L12 cathode	14	L13 cathode
15	+9V common anode power	16	+9V common anode power

The shield includes on-board current-limiting resistors for these LEDs so they don’t have to include their own.

It also adds J6 which exposes two identical I²C connections intended to drive alphanumeric displays such as the four-character Adafruit “back-pack” units. These have a ribbon cable pinout as follows:

Pin	Signal	Pin	Signal
1	V _{HI} (typ. +5V)	2	V _{IO} (logic voltage)
3	Ground	4	SDA
5	SCL	6	SCL
7	SDA	8	Ground
9	V _{IO}	10	V _{HI}

This pin arrangement allows you to wire it to an IDC ribbon cable connector, split the ribbon cable in two, sending each half to a different I²C display. Alternatively, you could wire this to a wire-to-board socket with discrete wires which can be bundled by column or sequentially.

Connectors for Legacy Boards

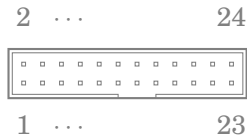
Connectors for Version 2.1 Boards

Power / RS-485 (8-pin terminal) [J0]

This is wired the same as J1 on the revision 3.2 boards.

Earlier Boards

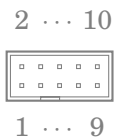
Matrix Control (24-pin ribbon cable) [J1]



For readerboards before version 2.1.0, this 24-position ribbon cable carries signals to directly drive the 64×8 LED matrix. The other end of this cable mates with J0 on the shield board. The pinout of its IDC header is:

1	D6	7	RCLK	13	+5 V DC	19	Gnd
2	D5	8	D2	14	+5 V DC	20	Gnd
3	D7	9	$\overline{G}$	15	+5 V DC	21	R2
4	D4	10	D1	16	+5 V DC	22	REN
5	SRCLK	11	$\overline{SRCLR}$	17	Gnd	23	R1
6	D3	12	D0	18	Gnd	24	R0

Discrete LEDs (10-pin ribbon cable) [J2]



For readerboards before version 2.1.0, this 10-position ribbon cable carries signals to directly drive the 64×8 LED matrix. The other end of the cable mates with J1 on the shield board. The pinout of its IDC header is:

1	GND	6	L6
2	GND	7	L2
3	L0	8	L5
4	L7	9	L3
5	L1	10	L4

Board Power (3-pin screw terminal) [J0]

This 3-position screw terminal block provides power to the readerboard. Note that the +5 V supply is also connected to pins 13–16, and Ground to pins 17–20 of J1, the only power required here is the +9 V input that drives the LEDs themselves.

1	+5 V DC in
2	GND
3	+9 V DC in

Shield Power (5-pin screw terminal) [Shield J2]

This 5-position screw terminal block accepts incoming +9 V DC power and ground on pins 4 and 3 respectively. It then provides +5 V DC, +9 V DC, and ground outputs on pins 1, 2, and 5 respectively to supply power to the main display board.

1	+5 V DC out
2	GND
3	GND
4	+9 V DC in
5	+9 V DC out

Shield RS-485 (6-pin screw terminal) [Shield J4]

This 6-position screw terminal block accepts incoming RS-485 signals A, B, and ground on pins 2, 1, and 3 respectively, and outputs the network signals A, B, and ground on pins 6, 5, and 4 respectively, to go on to the next device in the chain. If this is the last device, then nothing should be connected to pins 5 and 6. Instead, install jumper J5 which connects a 120Ω resistor across those terminals to terminate the network at that point.

1	B (Data In −)
2	A (Data In +)
3	GND
4	GND
5	B (Data Out −)
6	A (Data Out +)

CHAPTER 5

INSTALLING AND USING THE READERBOARD AND BUSYLIGHT UNITS

See, unlike most hackers, I get
little joy out of figuring out how
to install the latest toy.

—Jamie Zawinski

Power

The readerboard units require an external 9 V DC power supply in order to sustain the power requirements of all of the LEDs it uses. Always be sure that this power supply is connected and operating before plugging in the readerboard to USB. Otherwise, you may overload the USB port on the host computer and/or the Arduino microcontroller as it tries to power the readerboard by itself.

Similarly, busylight units are designed to be powered from the USB connection *or* from an external power source (such as a pair of power wires on the CAT6 cable used for RS-485 signals) but not both at the same time.

Readerboard PCB revisions starting with 3.4.1 introduced diode D520 to mitigate this issue, so the LEDs don't end up being powered by the Arduino. Busylight PCB revisions starting with 2.2.0 did the same with the introduction of D0.

Standalone Units

Networked Units

Compiling the Supporting Software

Setting up `rbserver`

Setting up Per-User Utilities

`calmond`

`micmond`

Manual Status Updates

Diagnostic Displays

There are a few situations where the readerboards and busylights use their light displays to convey information about their own status.

EEPROM Access

When a readerboard accesses its EEPROM, a small superscripted “*EE*” is displayed in the upper-right corner of the matrix until the matrix is cleared or overwritten. The color of the indicator indicates the type or outcome of the operation:

RGB Units	Monochrome Units	Meaning
green	on	reading
yellow	on	writing
flashing red	flashing	error

Protocol Errors

If the device receives garbled or otherwise incorrect commands over either USB or RS-485, it lights the top discrete status LED and alternately strobes the fourth and fifth from the bottom. On units with our standard color scheme, this is the white and both red LEDs.

These will be displayed until the next status LED command changes them.

POST

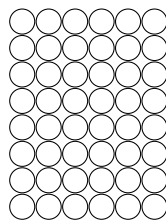
During power-on self-test when a readerboard is turned on, the top (usually white) LED is illuminated until the POST operation is completed.

FONT GLYPHS

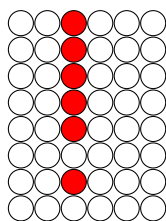
Helvetica is the sweatpants of
typefaces.

—John Boardley

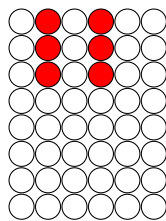
Font #0 standard.font



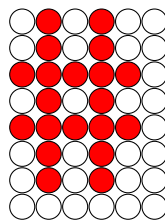
32 space



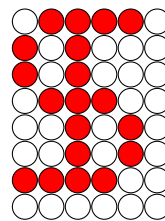
33 exclamation



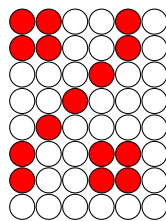
34 quotation mark



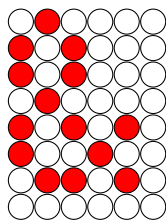
35 pound sign



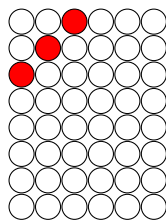
36 dollar sign



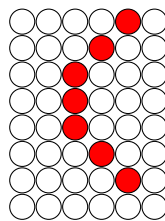
37 percent



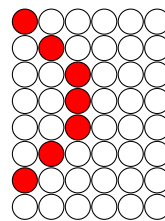
38 ampersand



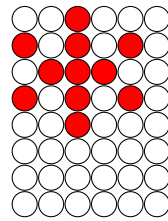
39 apostrophe



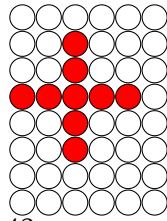
40 left paren



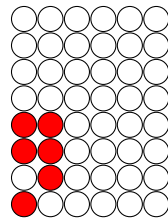
41 right paren



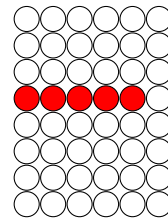
42 asterisk



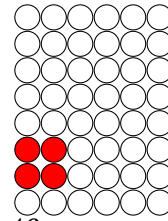
43 plus



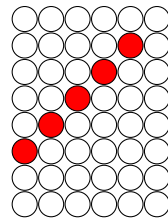
44 comma



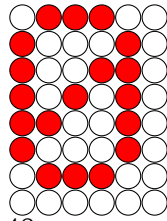
45 minus



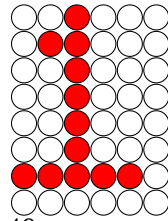
46 period



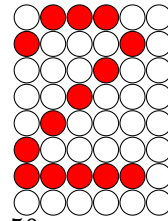
47 slash



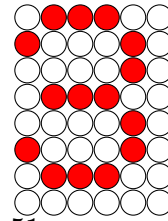
48 digit 0



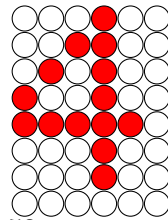
49 digit 1



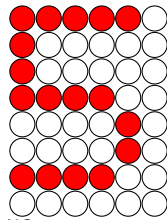
50 digit 2



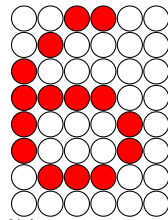
51 digit 3



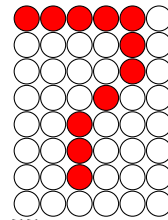
52 digit 4



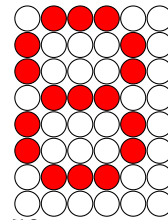
53 digit 5



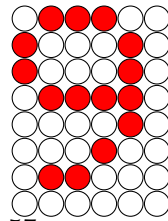
54 digit 6



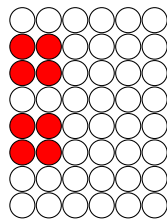
55 digit 7



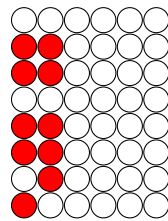
56 digit 8



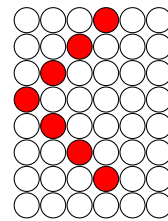
57 digit 9



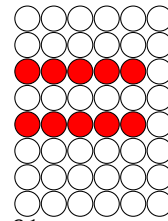
58 colon



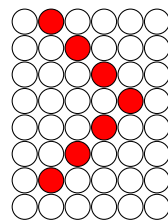
59 semi-colon



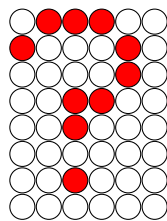
60 less than



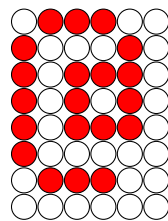
61 equals



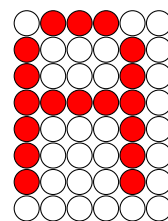
62 greater than



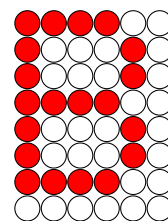
63 question mark



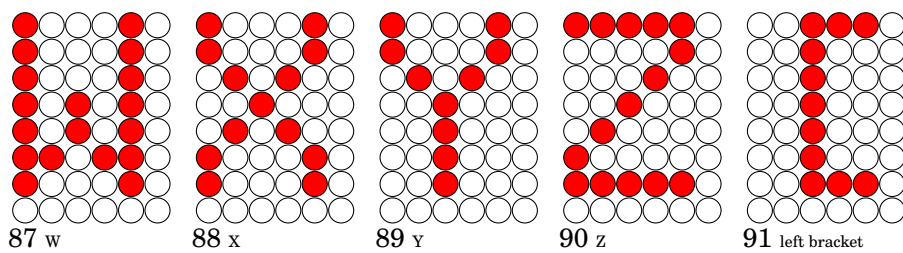
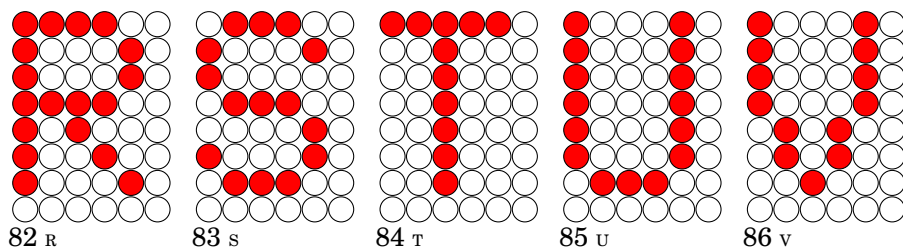
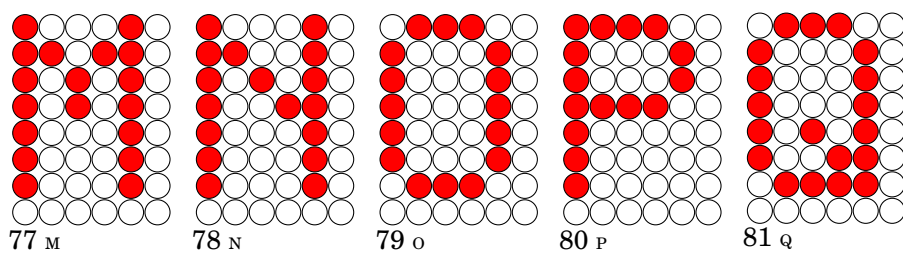
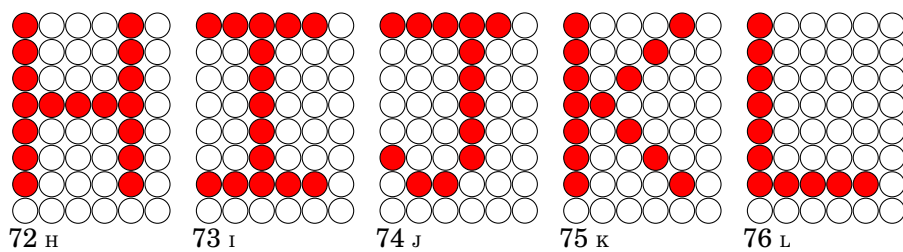
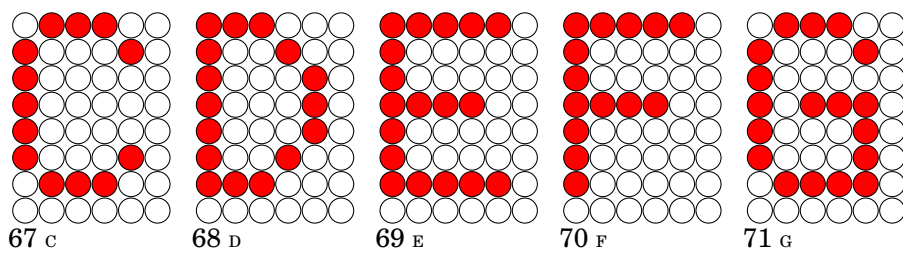
64 at-sign

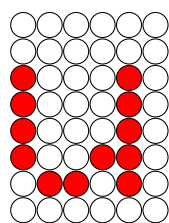


65 A

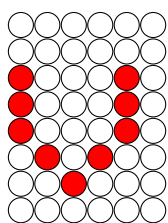


66 B

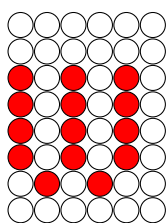




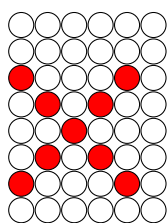
117 u



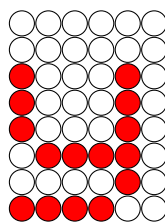
118 v



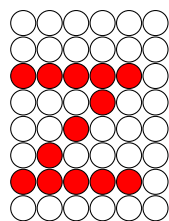
119 w



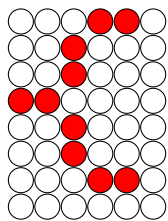
120 x



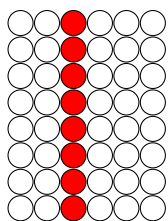
121 y



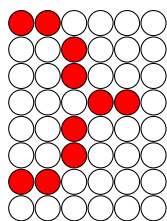
122 z



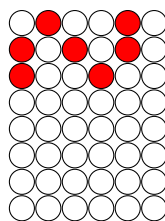
123 open brace



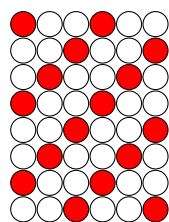
124 vertical bar



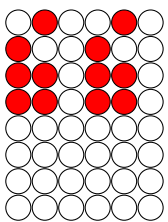
125 close brace



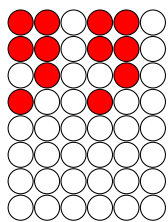
126 tilde



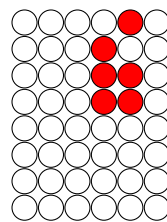
127 delete



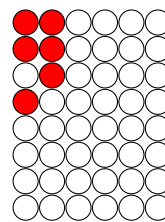
128 open dbl quote



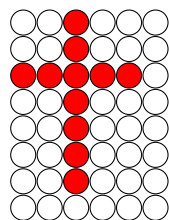
129 close dbl quote



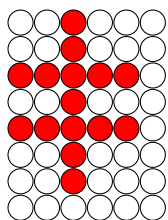
130 open sgl quote



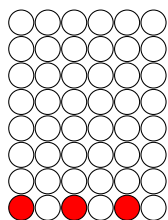
131 close sgl quote



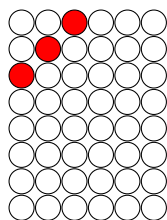
132 dagger



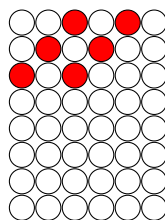
133 double dagger



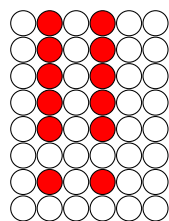
134 ellipsis



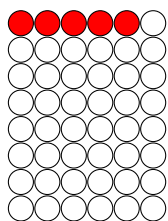
135 prime



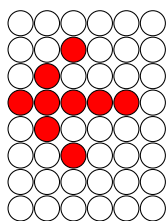
136 double prime



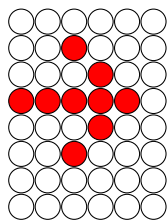
137 double !



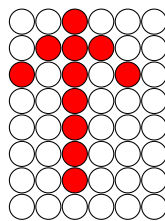
138 overline



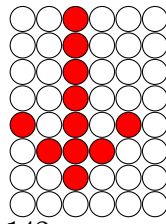
139 left arrow



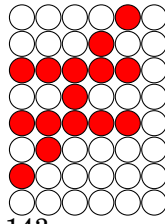
140 right arrow



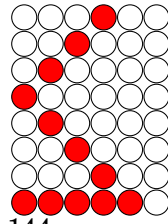
141 up arrow



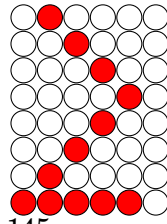
142 down arrow



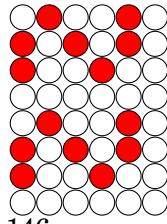
143 not equals



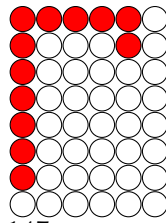
144 less or equal



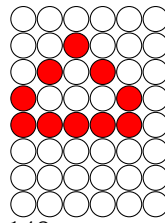
145 greater/equal



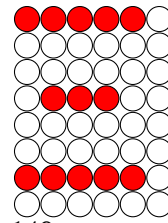
146 approximation



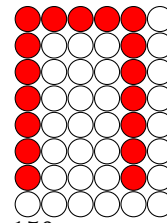
147 Gamma



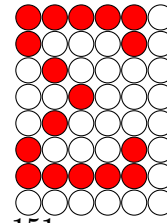
148 Delta



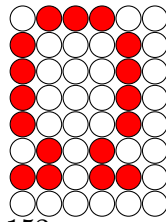
149 Xi



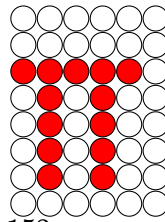
150 Pi



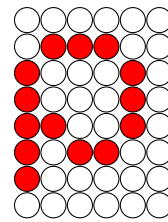
151 Sigma



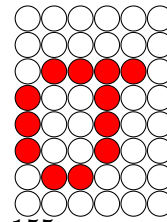
152 Omega



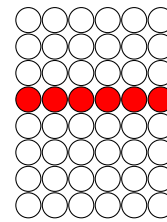
153 pi



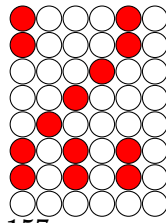
154 rho



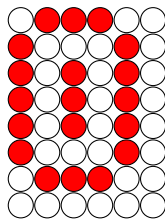
155 sigma



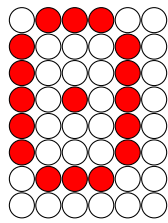
156 em dash



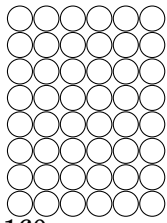
157 per mille



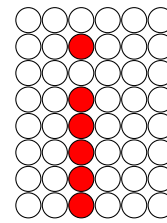
158 alt zero



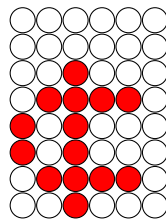
159 alt zero



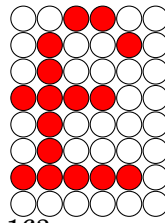
160 (unicode space)



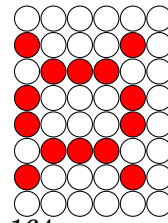
161 inverted !



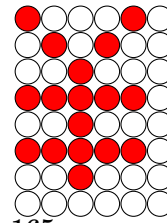
162 cents



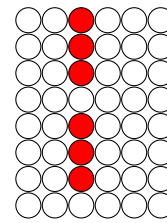
163 pounds



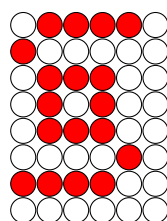
164 currency



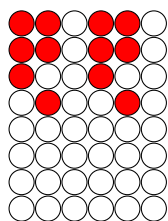
165 yen



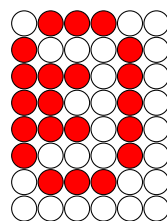
166 broken bar



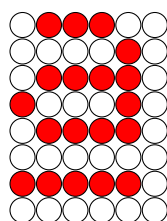
167 section



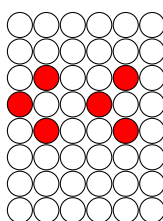
168 open dbl quote



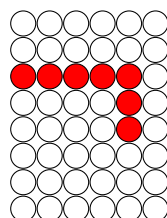
169 copyright



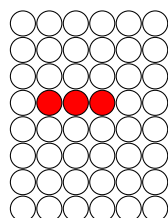
170 ª ordinal



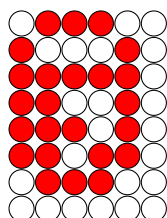
171 «



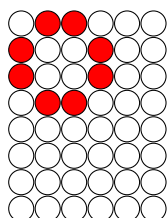
172 logical not



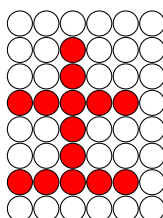
173 (uni hyphen)



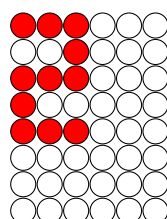
174 registered



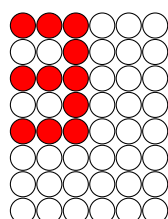
176 degrees



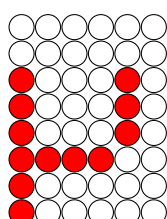
177 plusminus



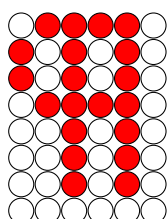
178 superscript 2



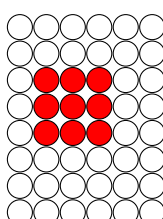
179 superscript 3



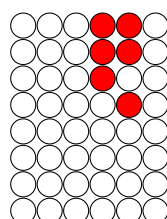
181 Greek mu



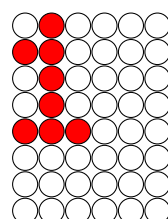
182 pilcrow



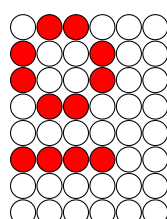
183 mid dot



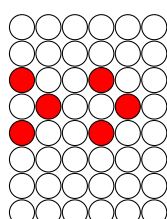
184 open sgl quote



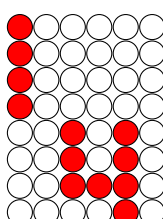
185 superscript 1



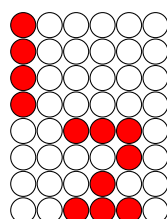
186 º ordinal



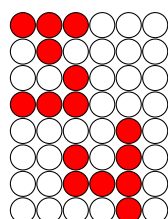
187 »



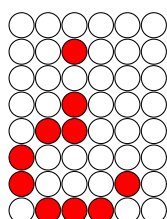
188 1/4



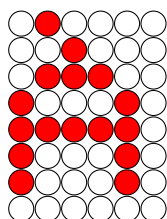
189 1/2



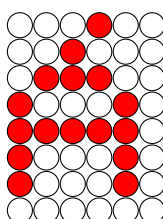
190 3/4



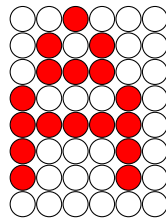
191 inverted ?



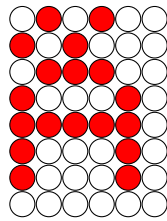
192 À grave



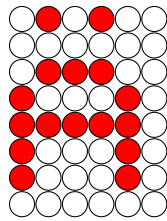
193 Á acute



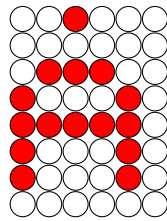
194 A circumflex



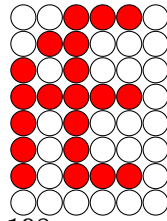
195 A tilde



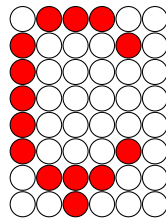
196 A umlaut



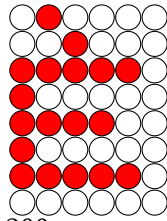
197 A circle



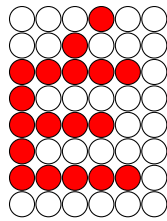
198 AE ligature



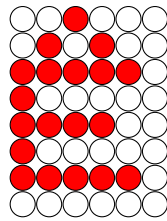
199 C cedilla



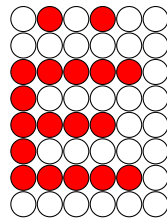
200 E grave



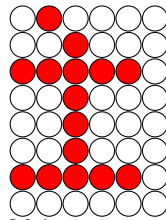
201 E acute



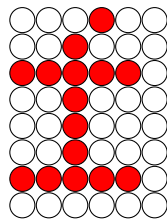
202 E circumflex



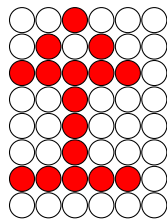
203 E umlaut



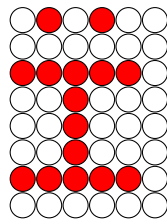
204 I grave



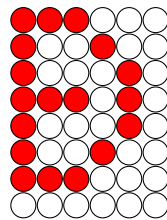
205 I acute



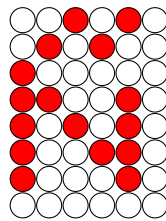
206 I circumflex



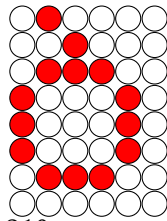
207 I umlaut



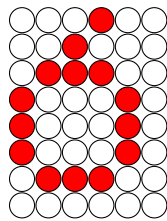
208 eth



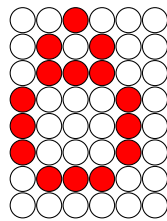
209 N tilde



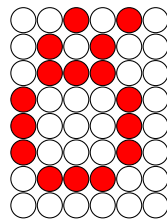
210 O grave



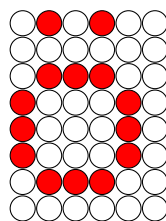
211 O acute



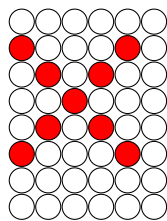
212 O circumflex



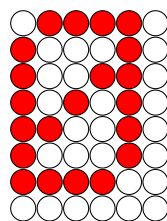
213 O tilde



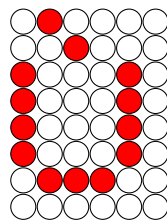
214 O umlaut



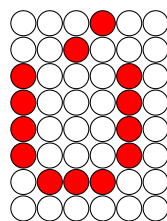
215 multiplication



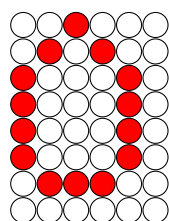
216 slashed O



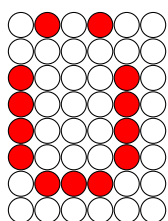
217 U grave



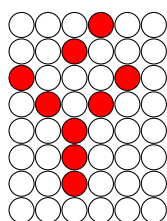
218 U acute



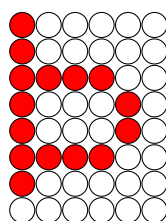
219 U circumflex



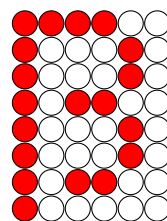
220 U umlaut



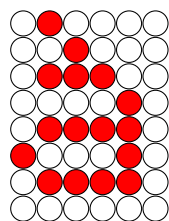
221 Y acute



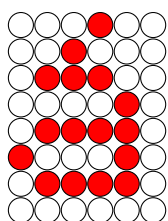
222 thorn



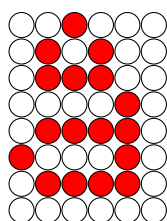
223 sharp S



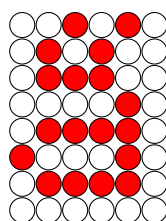
224 a grave



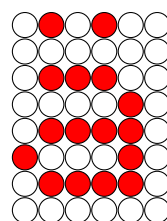
225 a acute



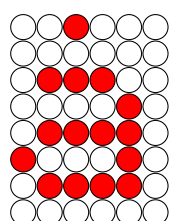
226 a circumflex



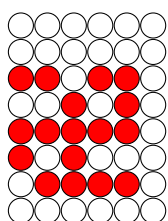
227 a tilde



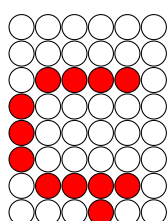
228 a umlaut



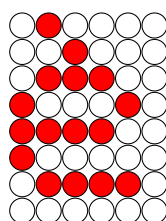
229 a circle



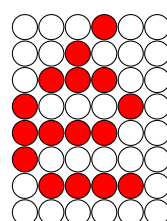
230 ae ligature



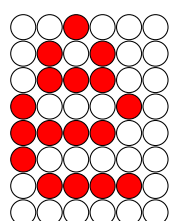
231 c cedilla



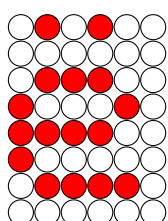
232 e grave



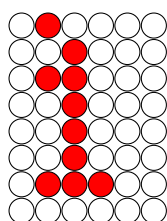
233 e acute



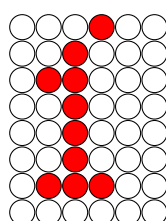
234 e circumflex



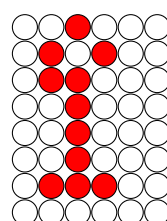
235 e umlaut



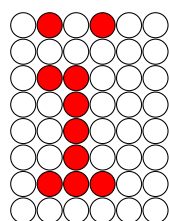
236 i grave



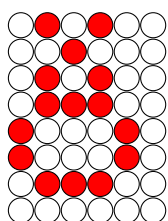
237 i acute



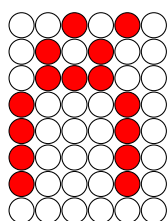
238 i circumflex



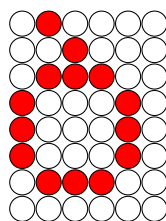
239 i umlaut



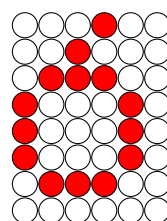
240 eth



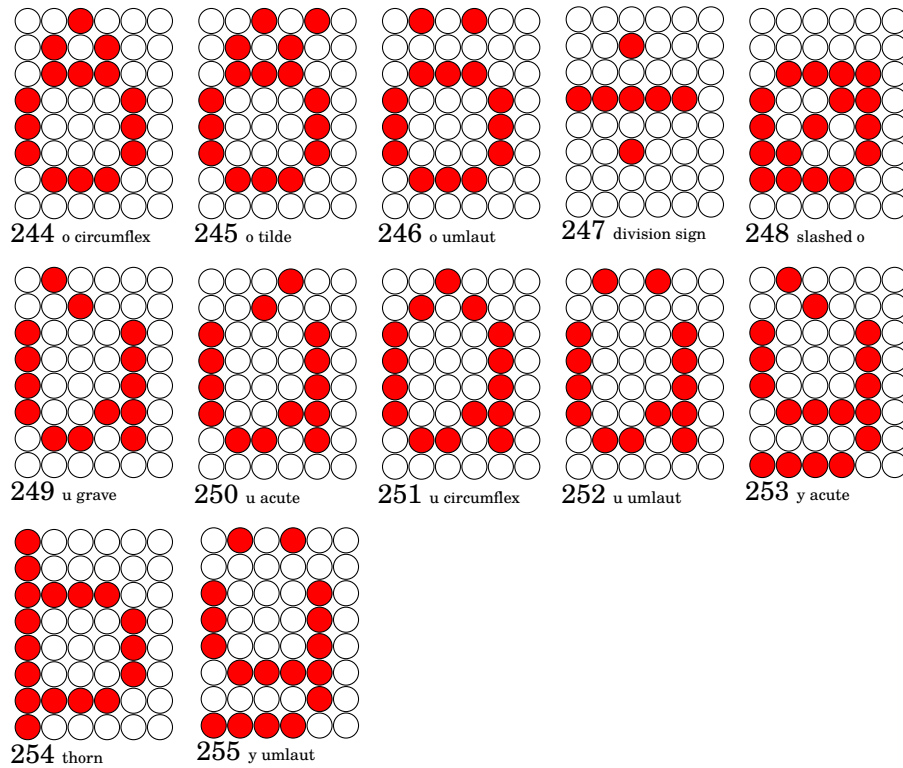
241 n tilde



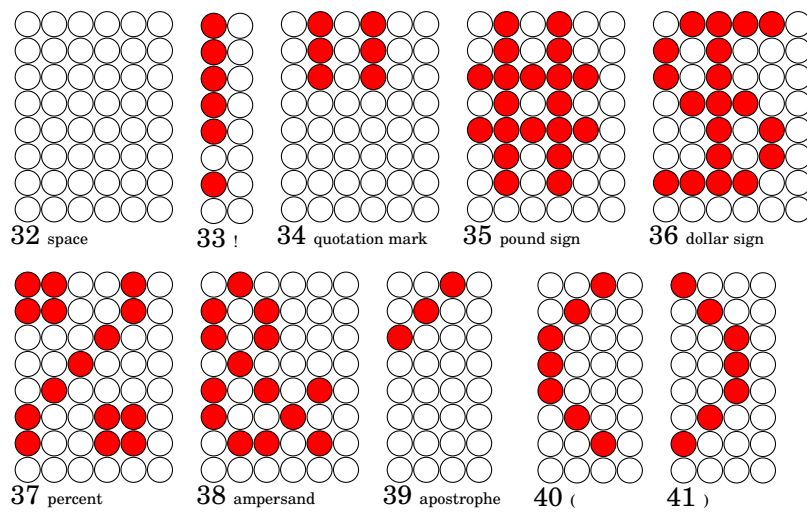
242 o grave

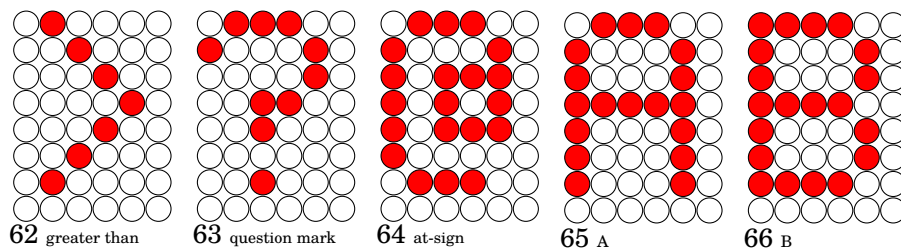
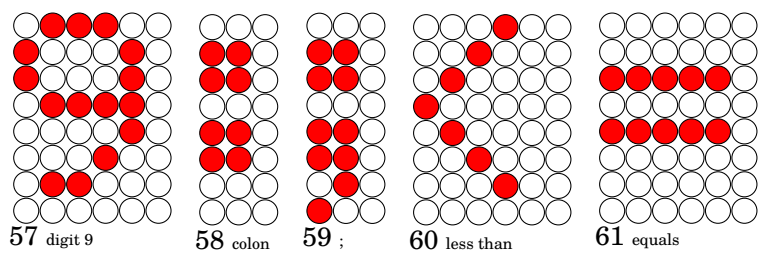
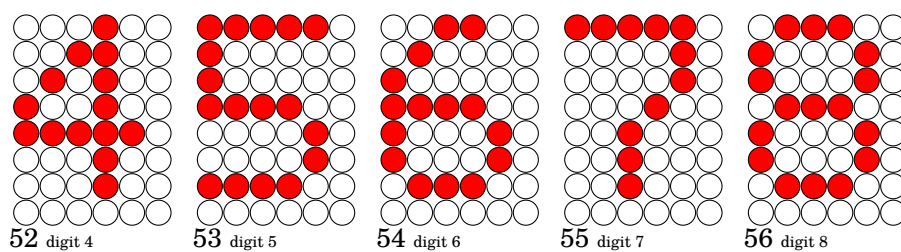
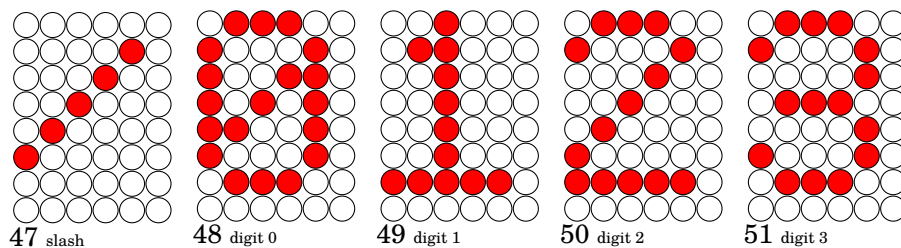
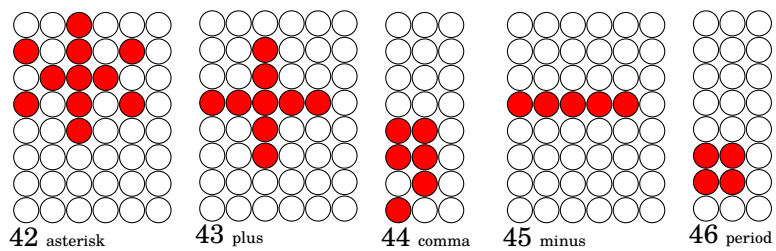


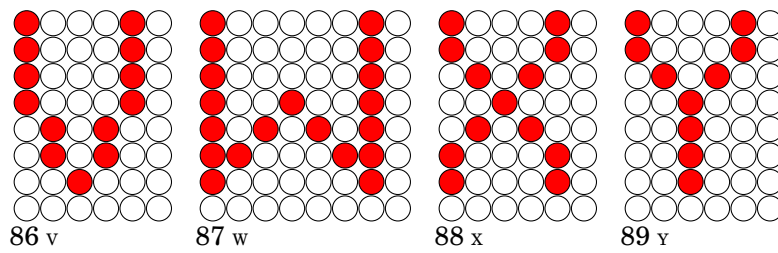
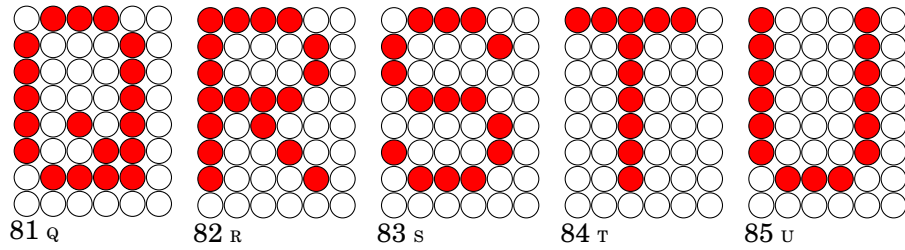
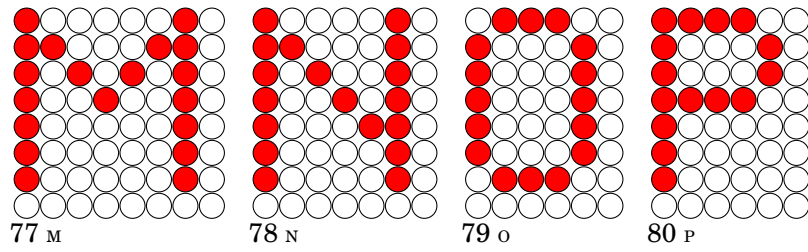
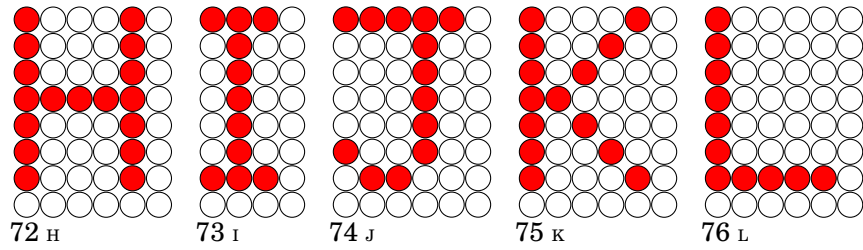
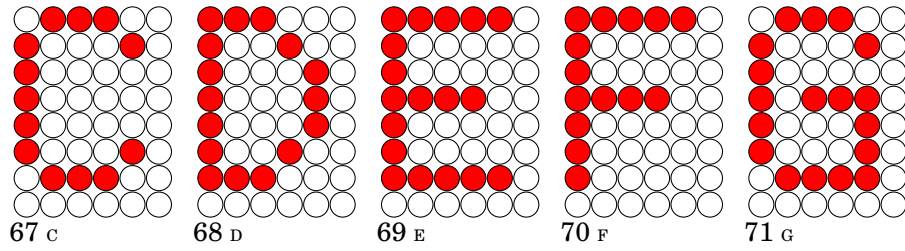
243 o acute

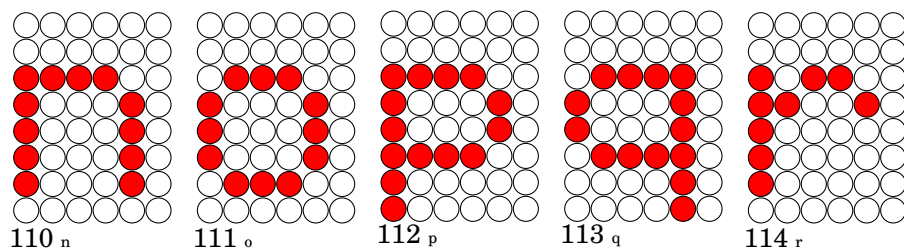
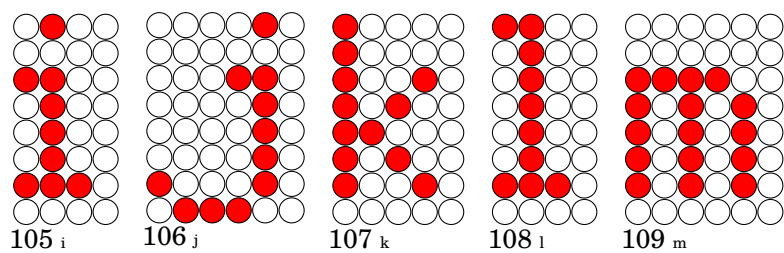
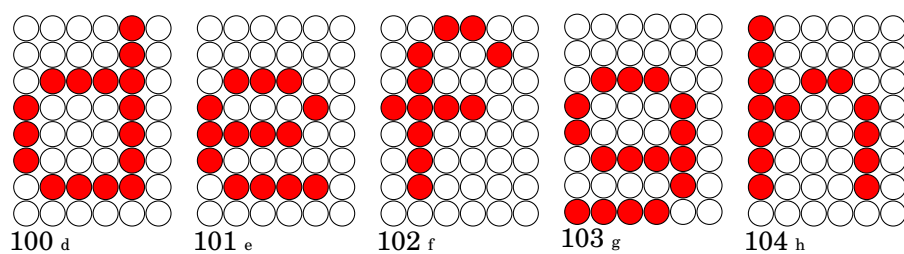
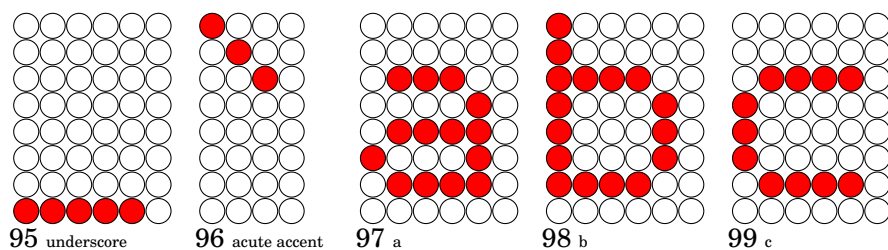
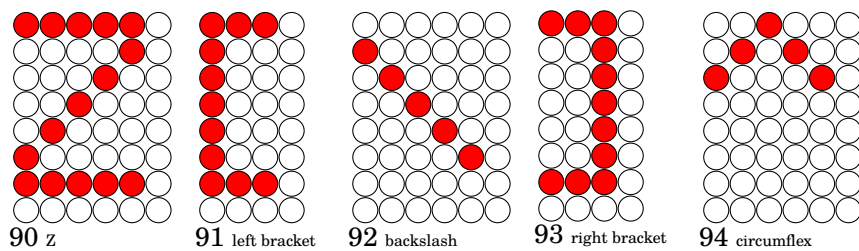


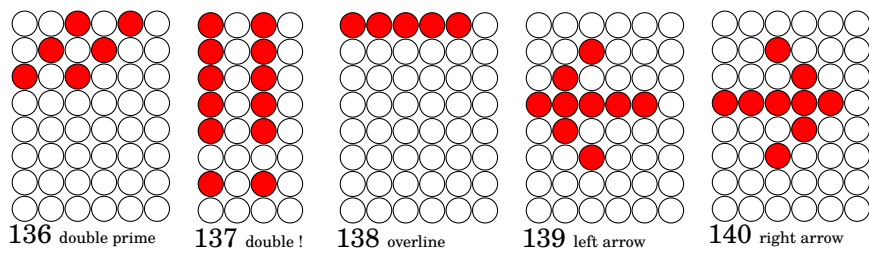
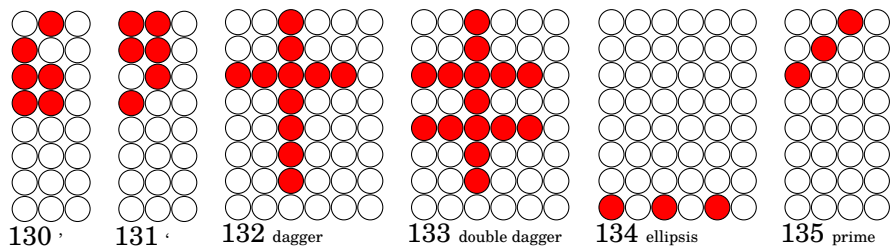
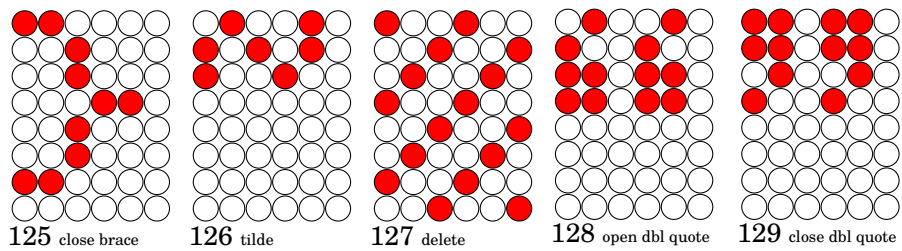
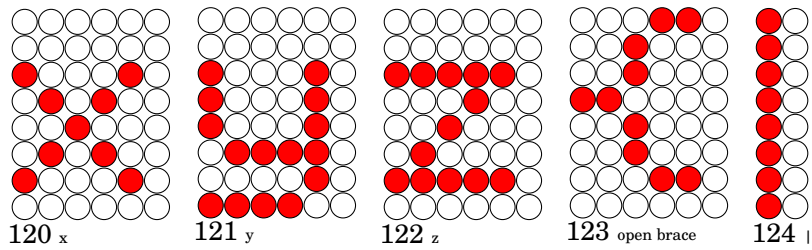
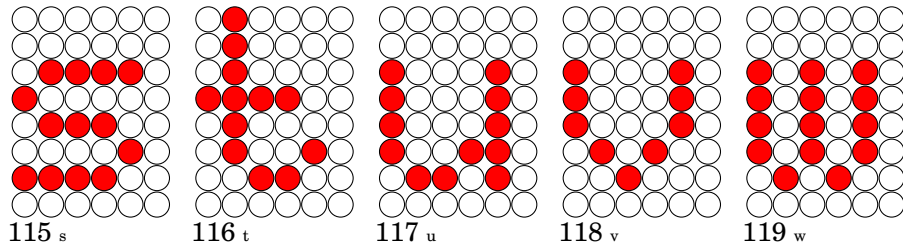
Font #1 standard_variable.font

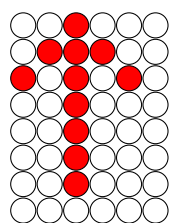




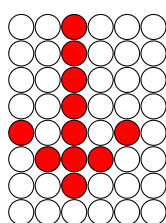




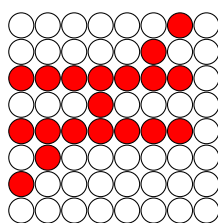




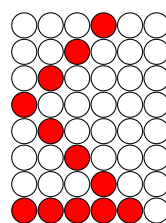
141 up arrow



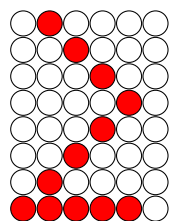
142 down arrow



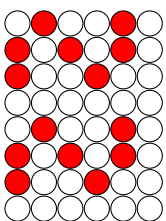
143 not equals



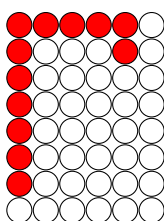
144 less or equal



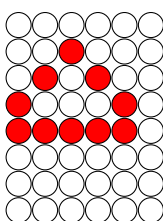
145 greater/equal



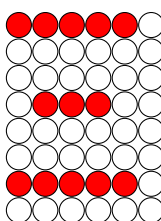
146 approximation



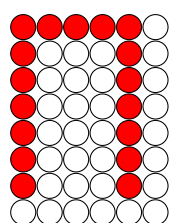
147 Gamma



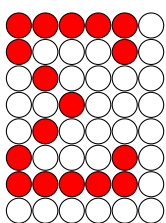
148 Delta



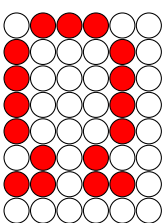
149 Xi



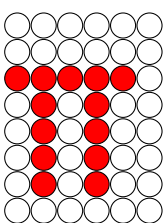
150 Pi



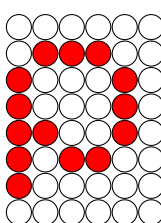
151 Sigma



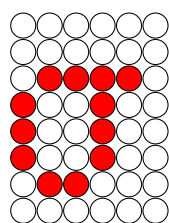
152 Omega



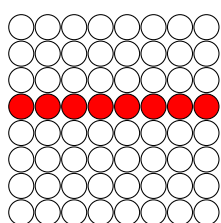
153 pi



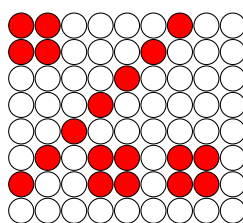
154 rho



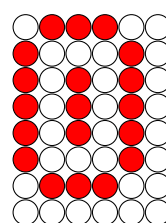
155 sigma



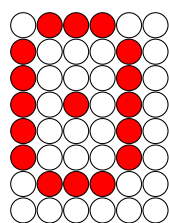
156 em dash



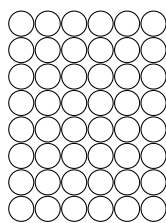
157 per mille



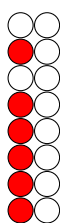
158 alt zero



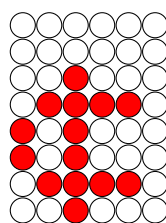
159 alt zero



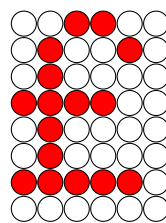
160 (unicode space)



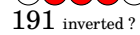
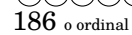
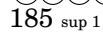
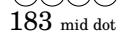
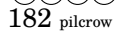
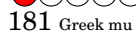
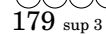
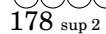
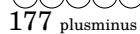
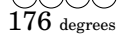
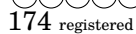
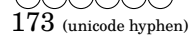
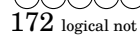
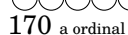
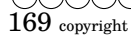
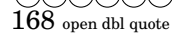
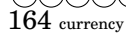
161

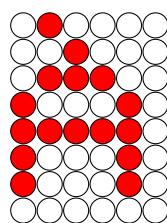


162 cents

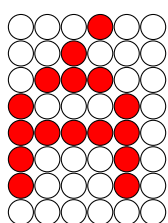


163 pounds

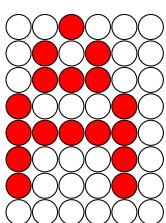




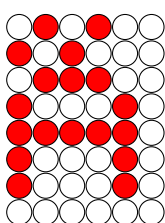
192 A grave



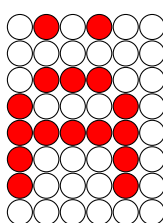
193 A acute



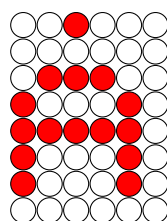
194 A circumflex



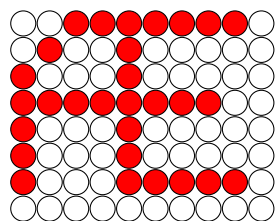
195 A tilde



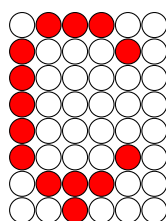
196 A umlaut



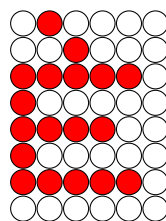
197 A circle



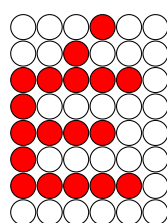
198 AE ligature



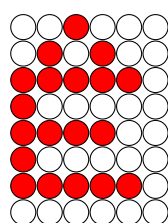
199 C cedilla



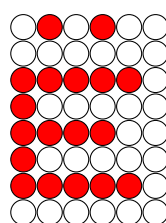
200 E grave



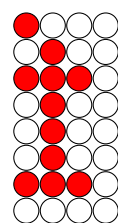
201 E acute



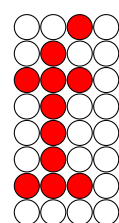
202 E circumflex



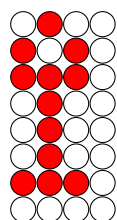
203 E umlaut



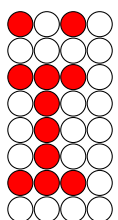
204 I grave



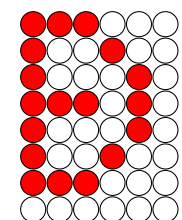
205 I acute



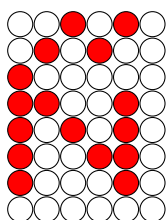
206 I circ



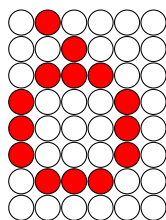
207 I umlaut



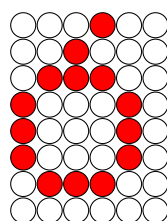
208 eth



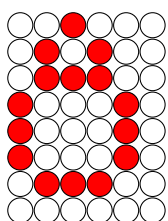
209 N tilde



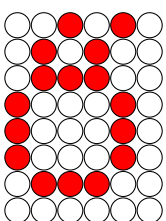
210 O grave



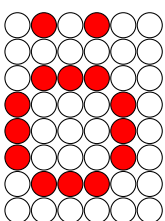
211 O acute



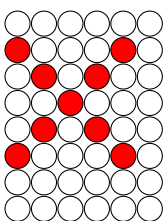
212 O circumflex



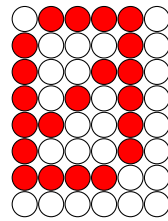
213 O tilde



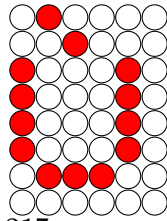
214 O umlaut



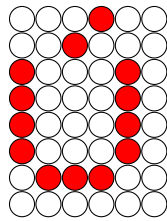
215 multiplication



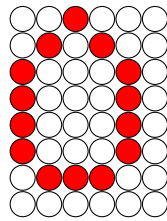
216 slashed O



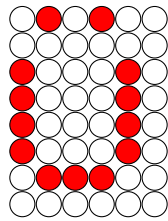
217 U grave



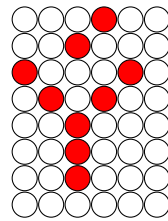
218 U acute



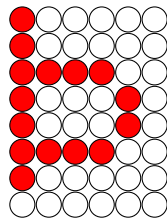
219 U circumflex



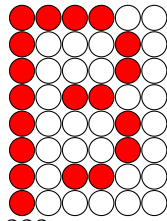
220 U umlaut



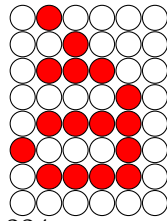
221 Y acute



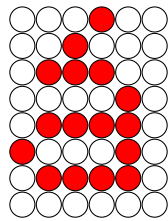
222 thorn



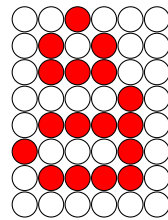
223 sharp S



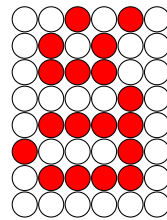
224 a grave



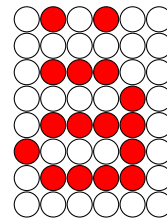
225 a acute



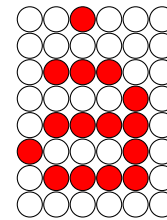
226 a circumflex



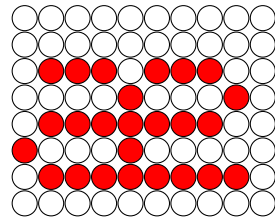
227 a tilde



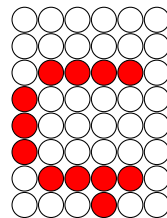
228 a umlaut



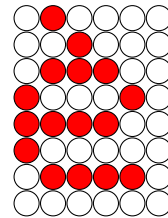
229 a circle



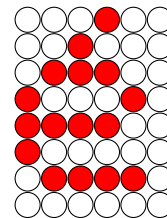
230 ae ligature



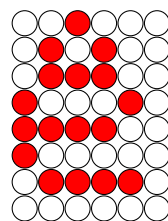
231 c cedilla



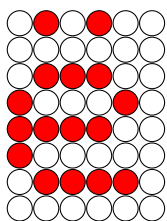
232 e grave



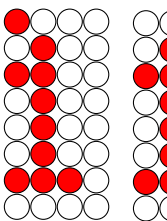
233 e acute



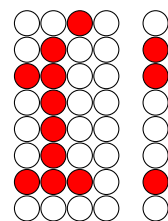
234 e circumflex



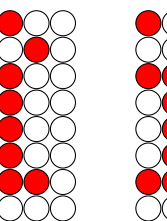
235 e umlaut



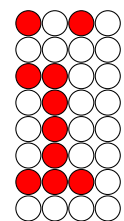
236 i grave



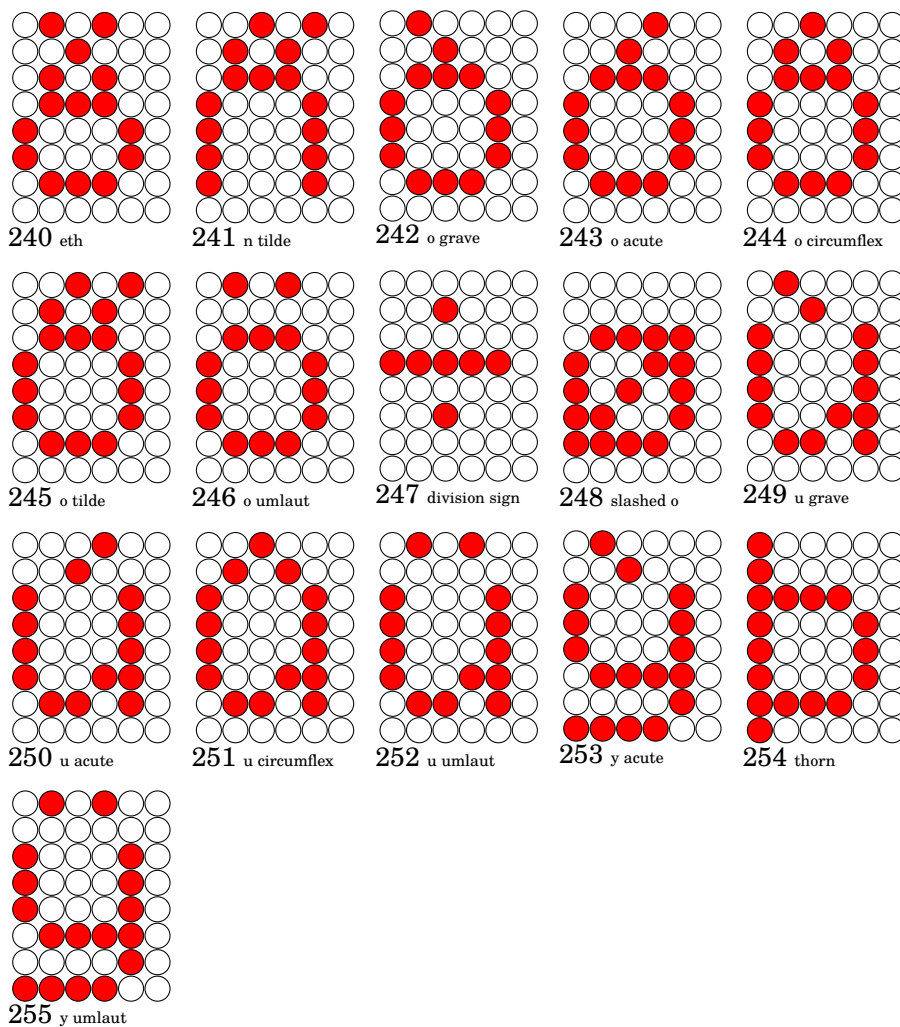
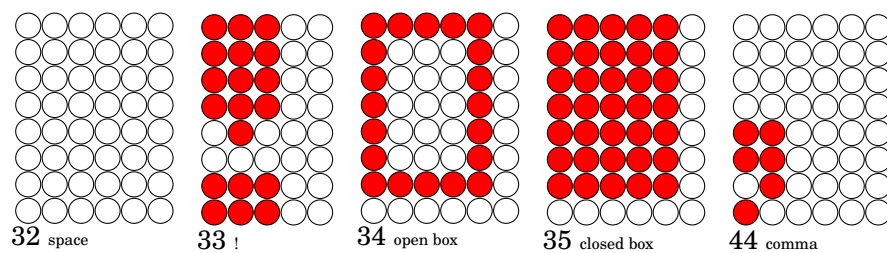
237 i acute

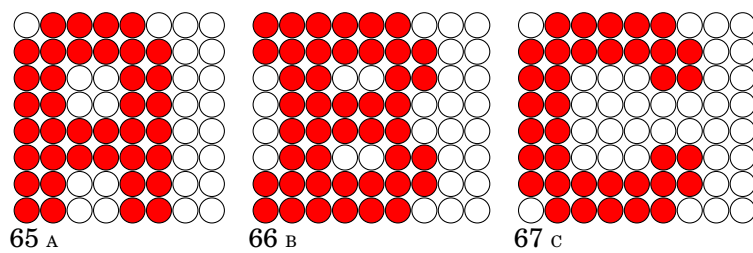
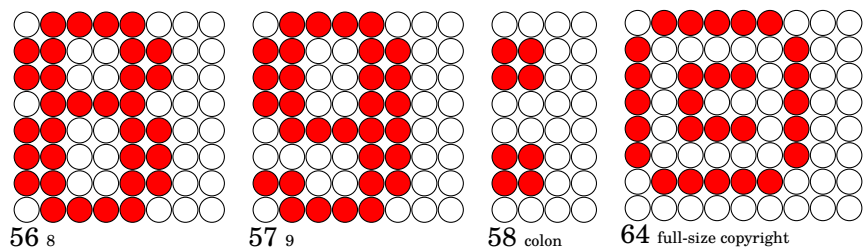
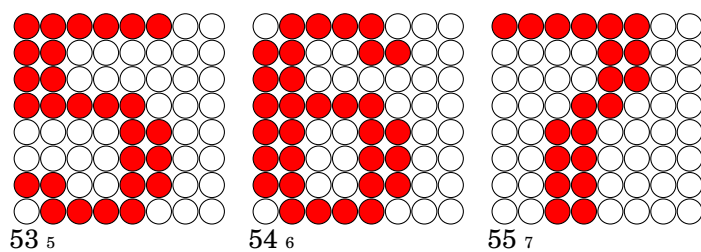
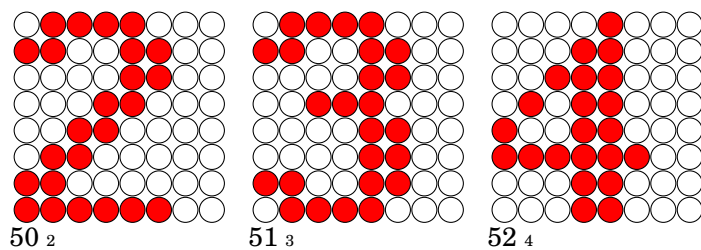
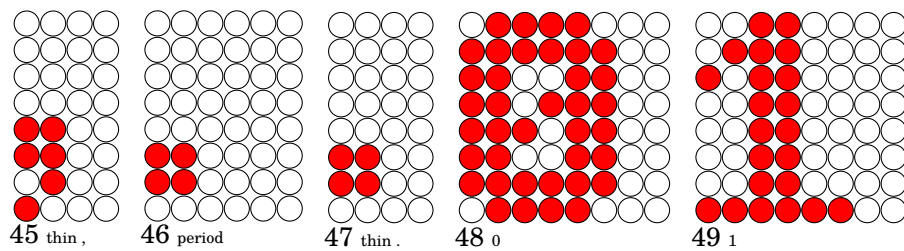


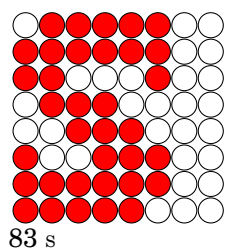
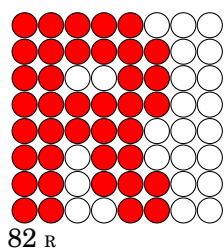
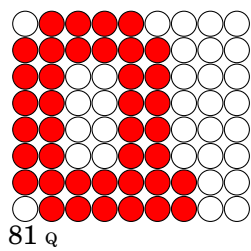
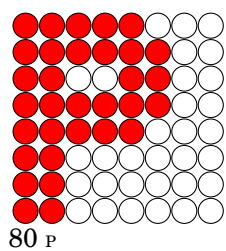
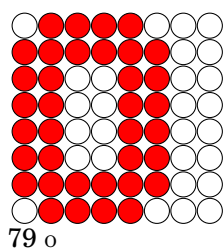
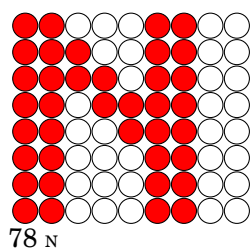
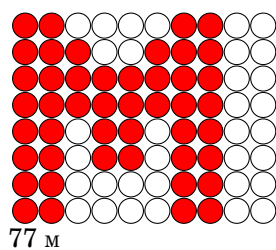
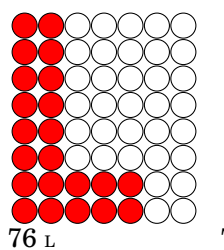
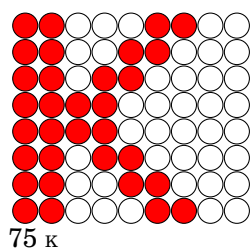
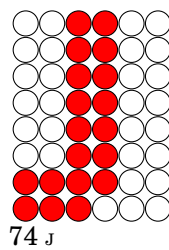
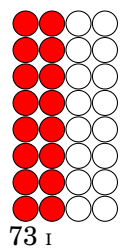
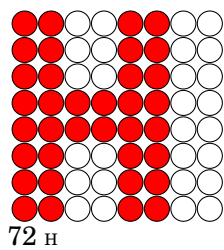
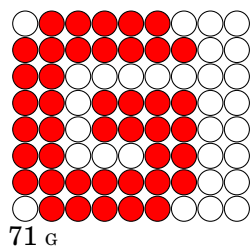
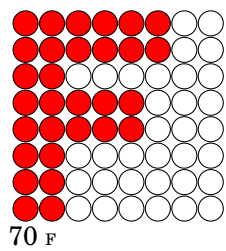
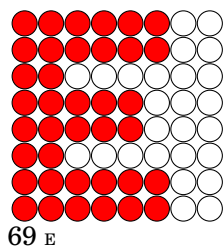
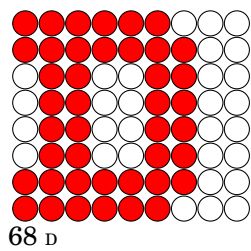
238 i circumflex

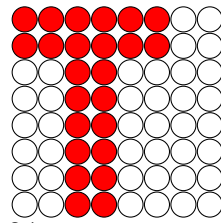


239 i umlaut

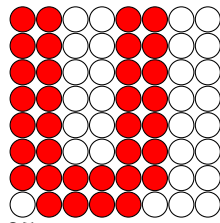
**Font #2** symbol.font



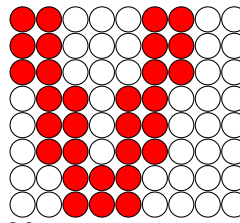




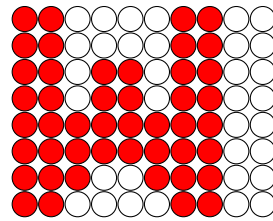
84 t



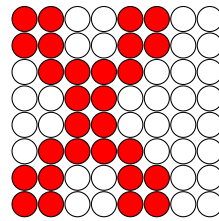
85 u



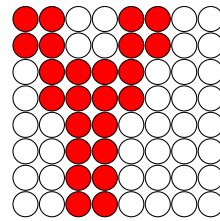
86 v



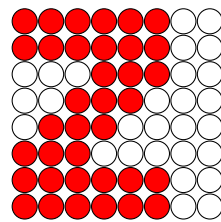
87 w



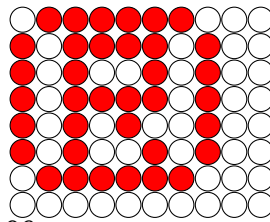
88 x



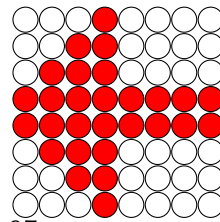
89 y



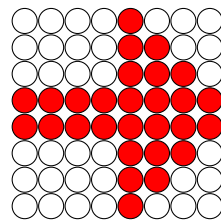
90 z



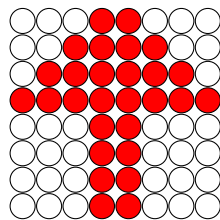
96 full-size registered



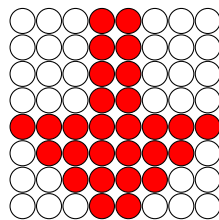
97 large left arrow



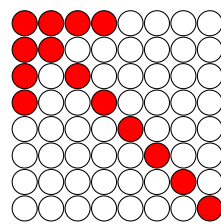
98 large right arrow



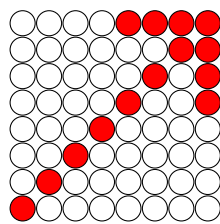
99 large up arrow



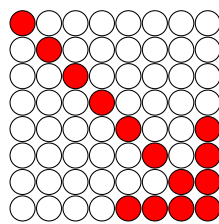
100 large down arrow



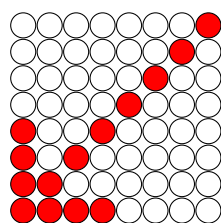
101 NW arrow



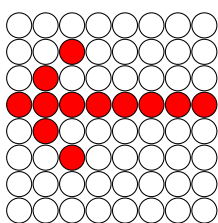
102 NE arrow



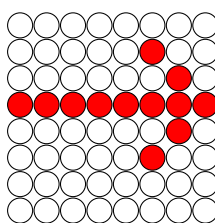
103 SE arrow



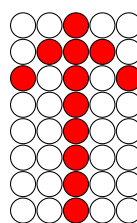
104 SW arrow



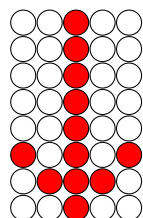
105 W arrow



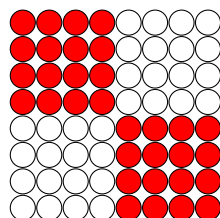
106 E arrow



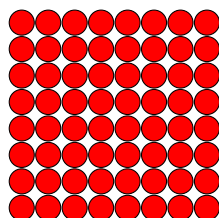
107 N arrow



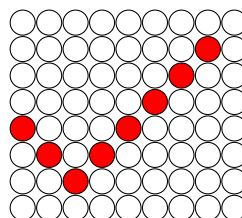
108 S arrow



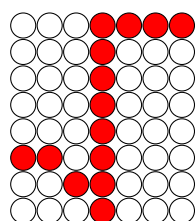
109 checkerboard



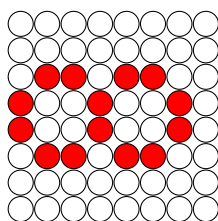
110 solid block



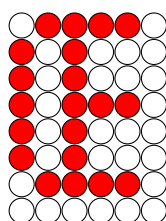
112 check



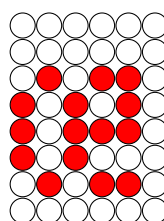
113 radical



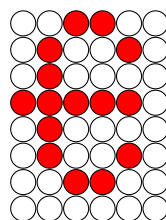
114 infinity



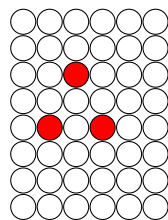
115 OE



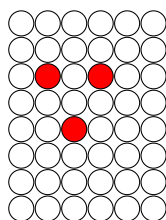
116 oe



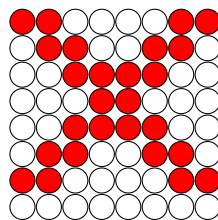
117 Euro



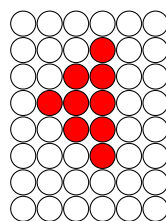
118 three dots up



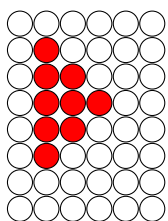
119 three dots down



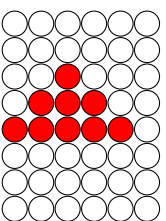
120 large cross



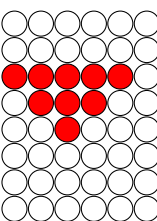
121 triangle left



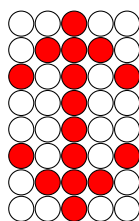
122 triangle right



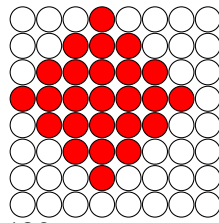
123 triangle up



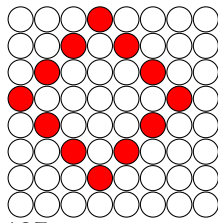
124 triangle down



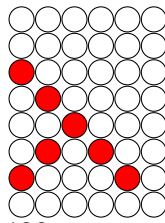
125 updown arrow



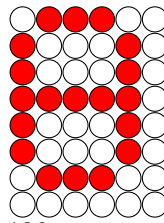
126 filled diamond



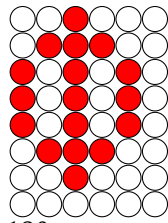
127 unfilled diamond



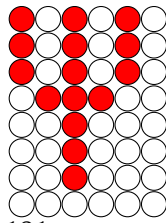
128 lambda



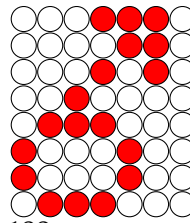
129 Theta



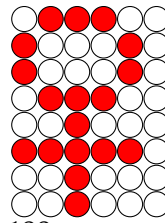
130 Phi



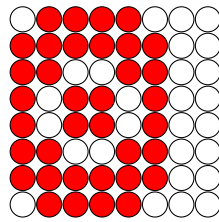
131 Psi



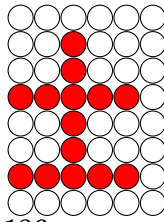
132 male



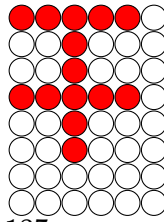
133 female



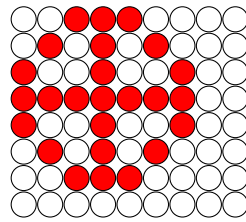
135 alt zero



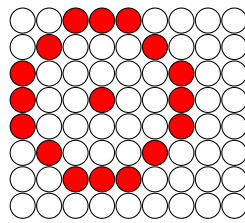
136 plus minus



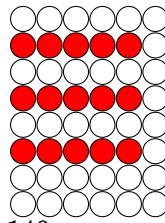
137 minus plus



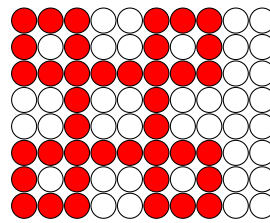
138 earth



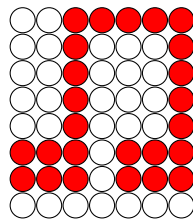
139 sun



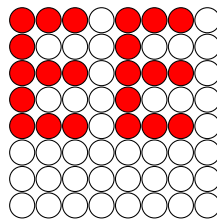
140 equiv



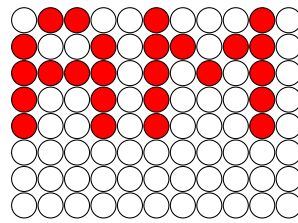
141 clover



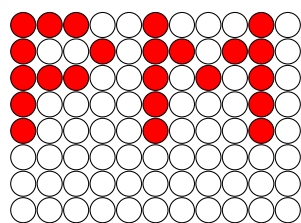
142 notes



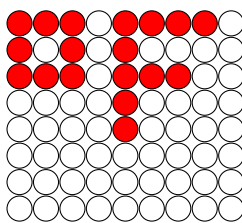
143 EE



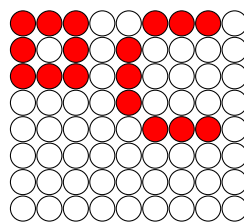
144 AM



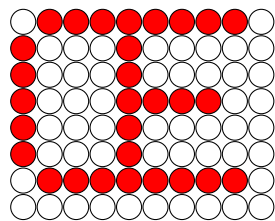
145 PM



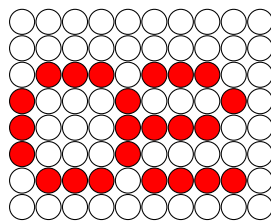
146 degrees F



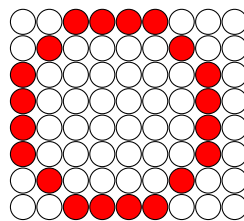
147 degrees C



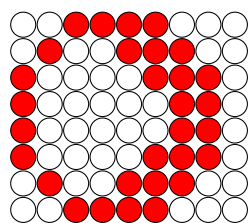
148 wide OE



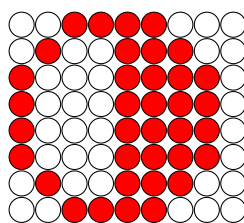
149 wide oe



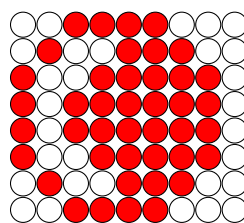
152 new moon



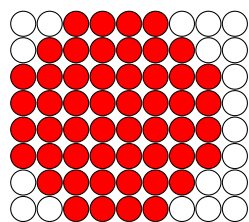
153 waxing crescent



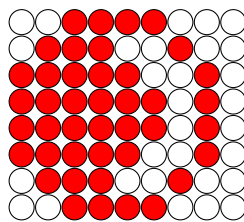
154 first quarter



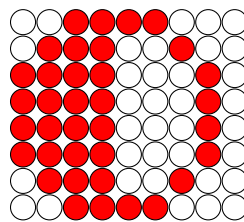
155 waxing gibbous



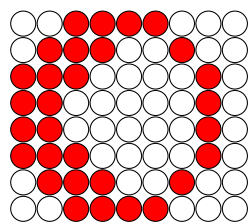
156 full moon



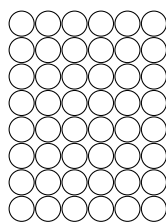
157 waning gibbous



158 third quarter



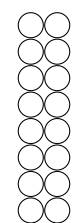
159 waning crescent



160 (unicode space)



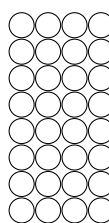
161



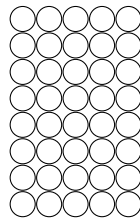
162



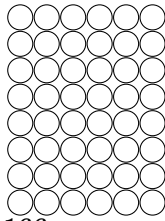
163



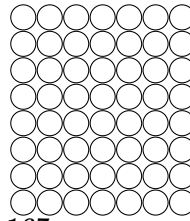
164 TS4



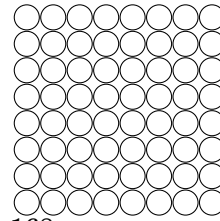
165 TS5



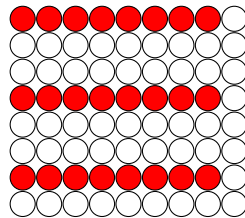
166 thin space 6



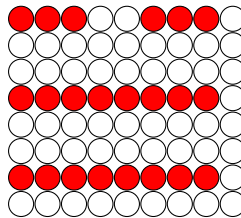
167 wide space 7



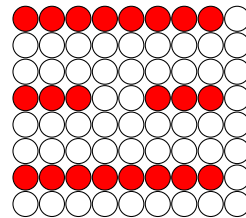
168 wide space 8



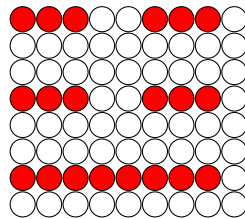
176 trigram 0



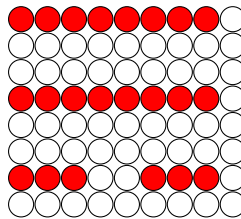
177 trigram 1



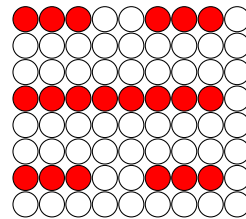
178 trigram 2



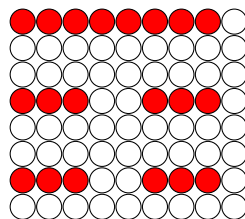
179 trigram 3



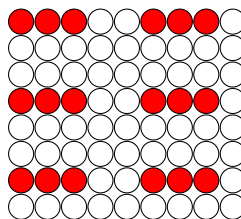
180 trigram 4



181 trigram 5

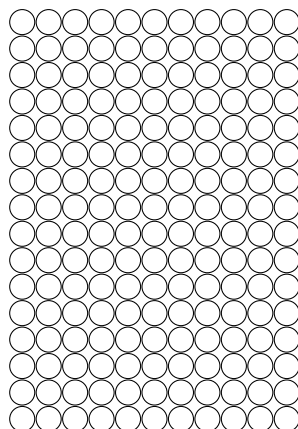


182 trigram 6

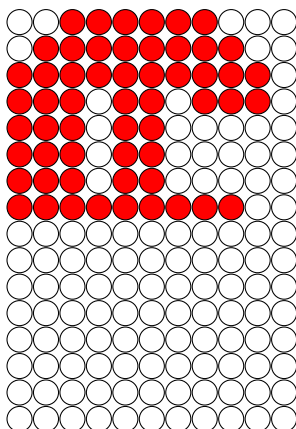


183 trigram 7

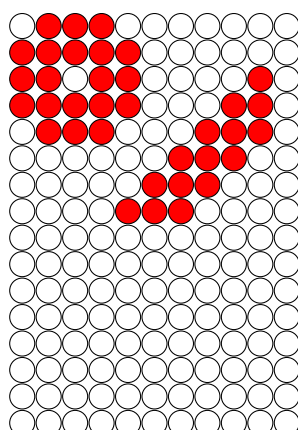
Font #3 double.font



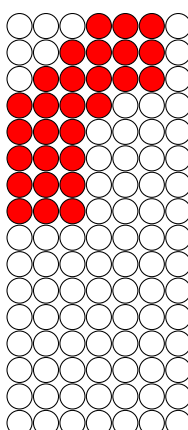
32 space



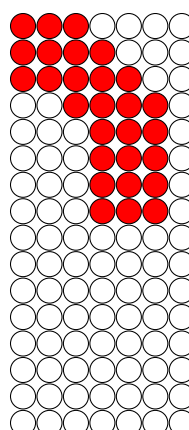
36 dollar sign



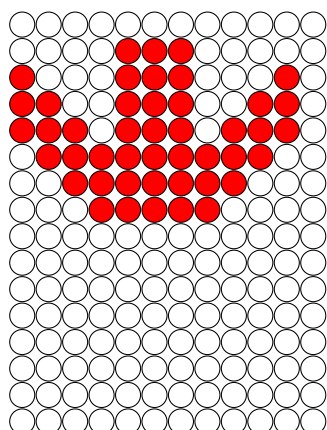
37 percent



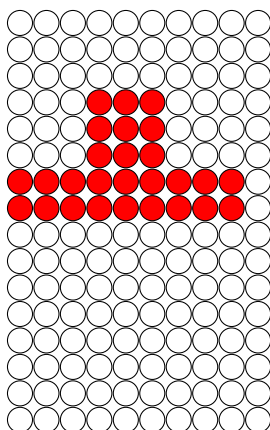
40 left parenthesis



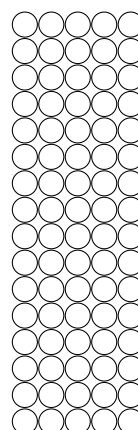
41 right parenthesis



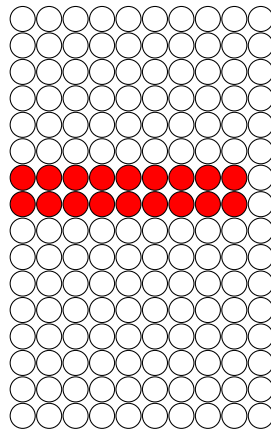
42 asterisk



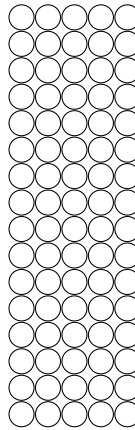
43 plus



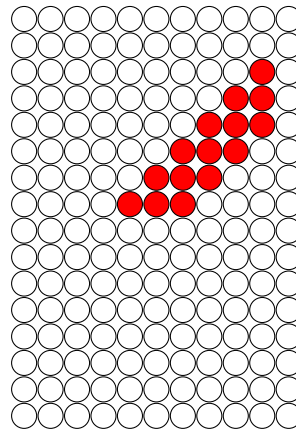
44 comma



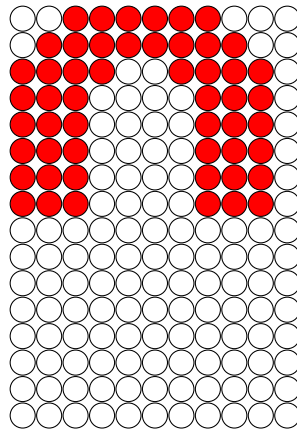
45 minus



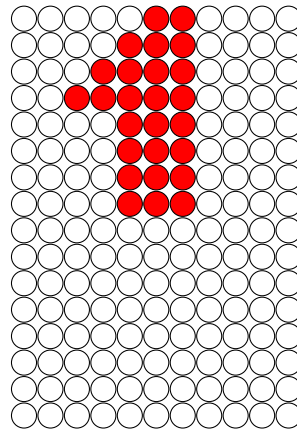
46 period



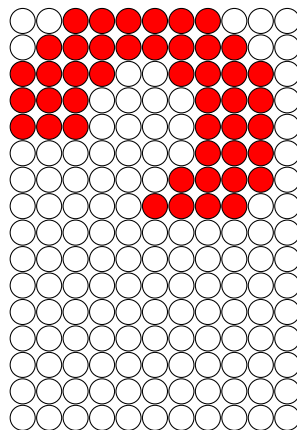
47 slash



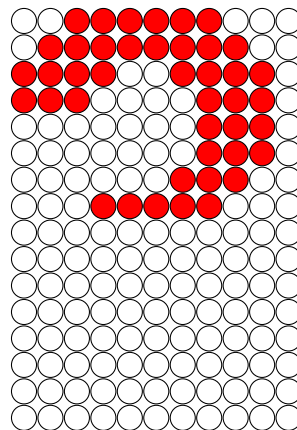
48 digit 0



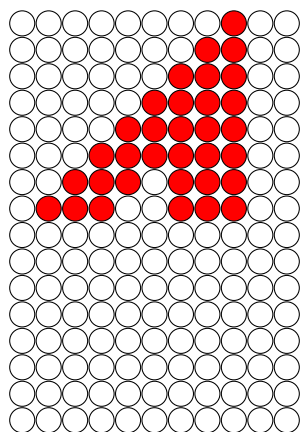
49 digit 1



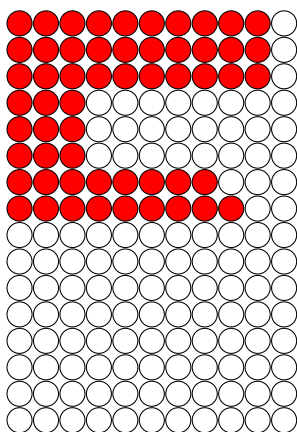
50 digit 2



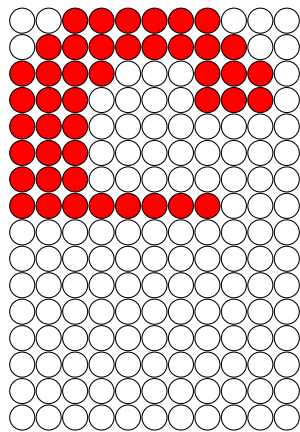
51 digit 3



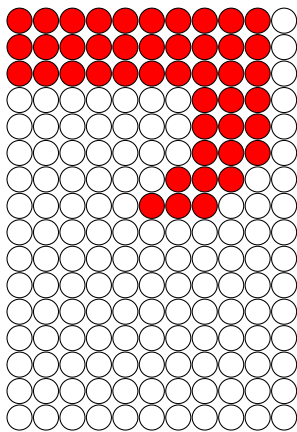
52 digit 4



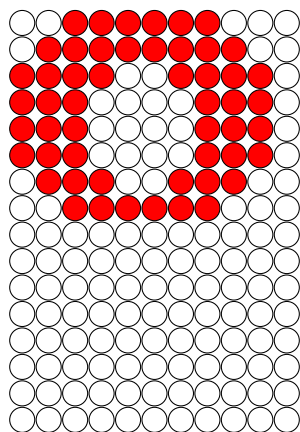
53 digit 5



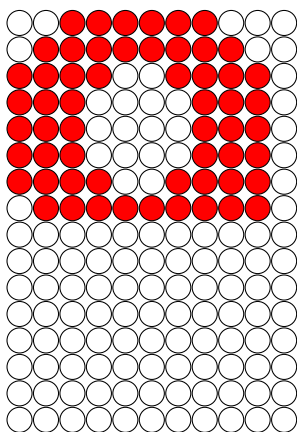
54 digit 6



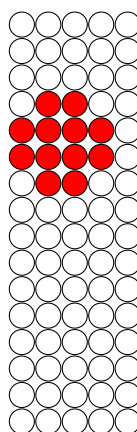
55 digit 7



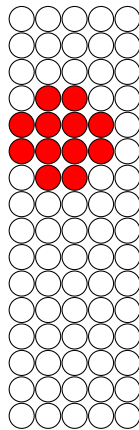
56 digit 8



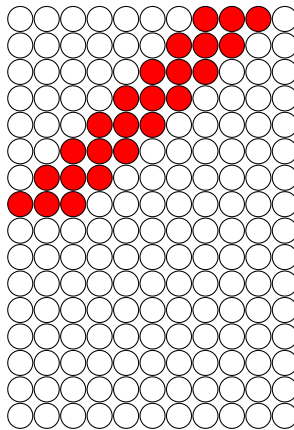
57 digit 9



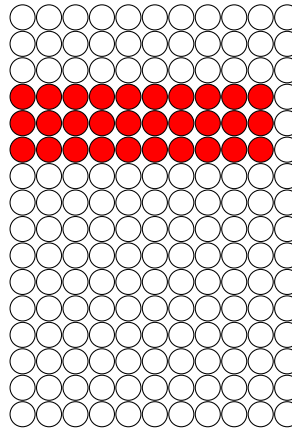
58 colon



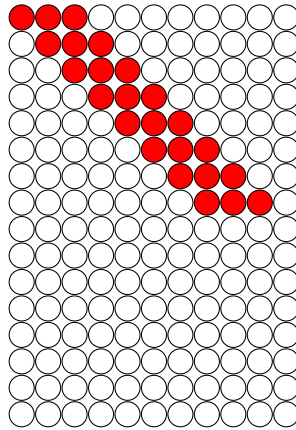
59 semi-colon



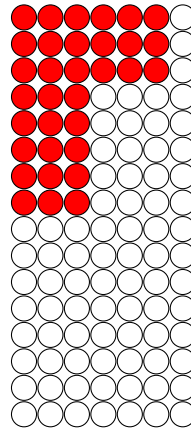
60 less than



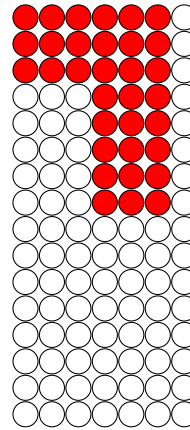
61 equals



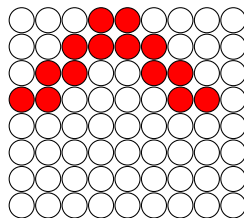
62 greater than



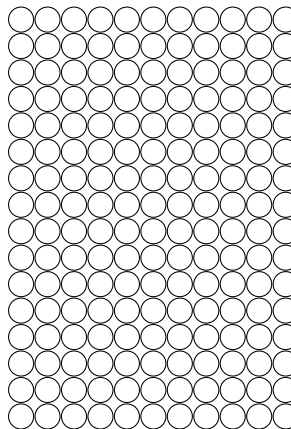
91 left square bracket



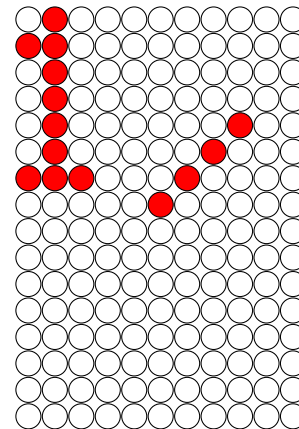
93 right square bracket



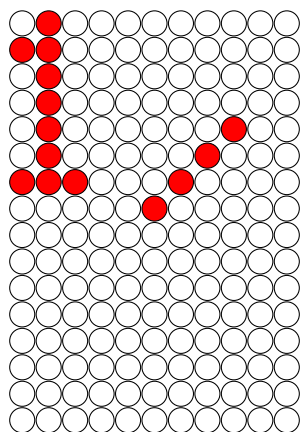
94 circumflex



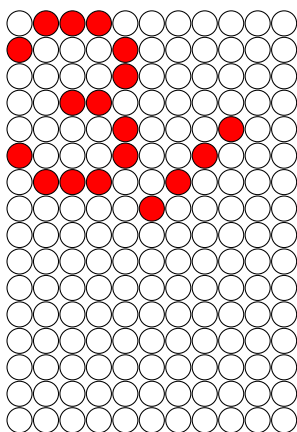
95 underscore



188 1/4

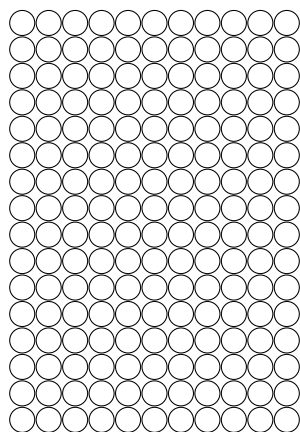


189 1/2

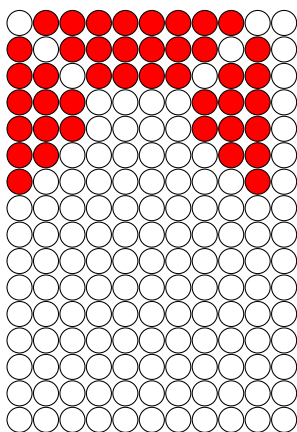


190 3/4

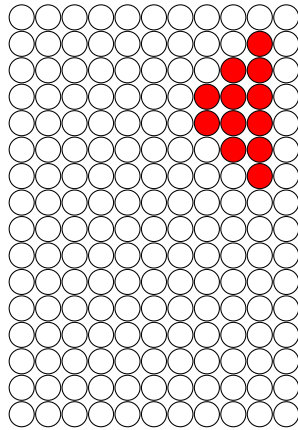
Font #4 led16.font



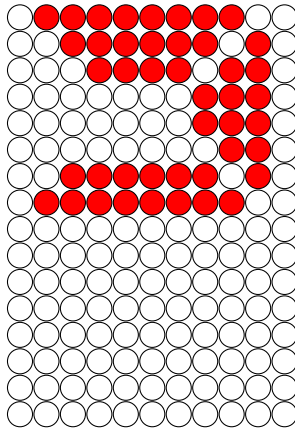
32 space



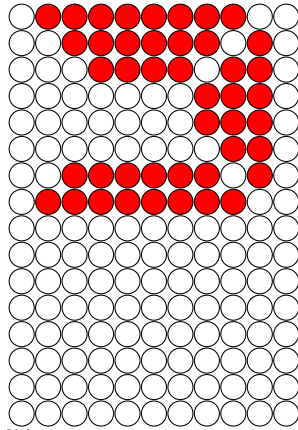
48 digit 0



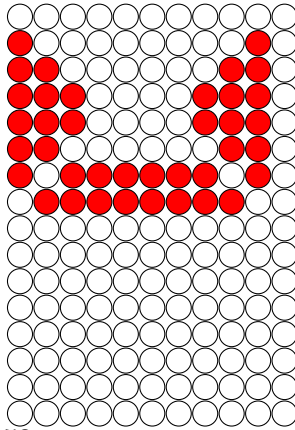
49 digit 1



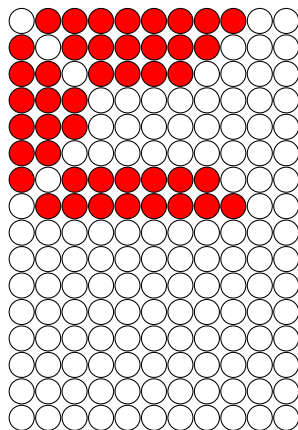
50 digit 2



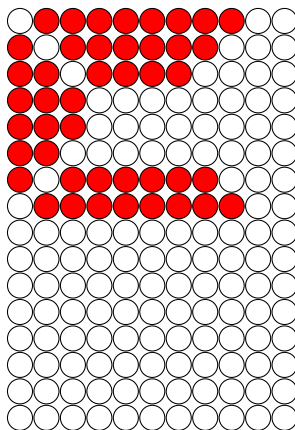
51 digit 3



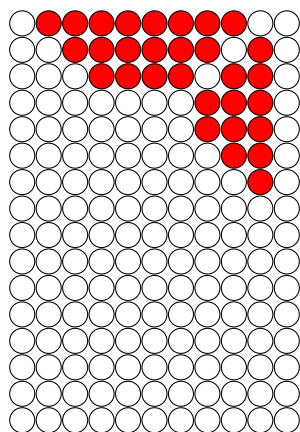
52 digit 4



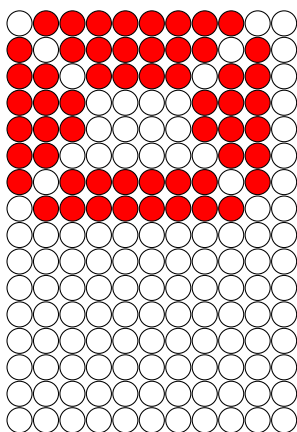
53 digit 5



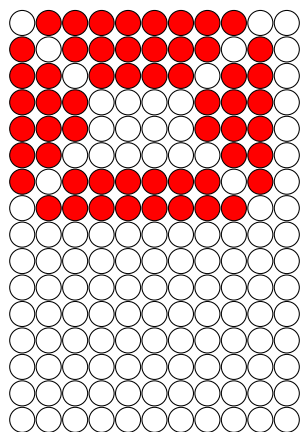
54 digit 6



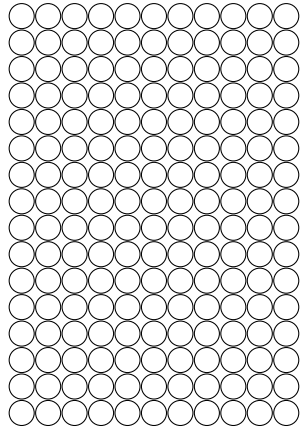
55 digit 7



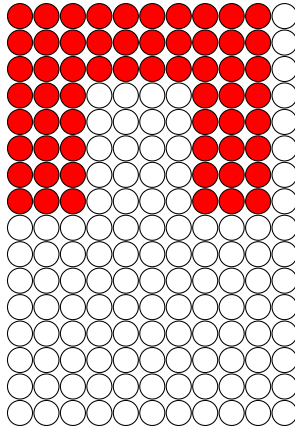
56 digit 8



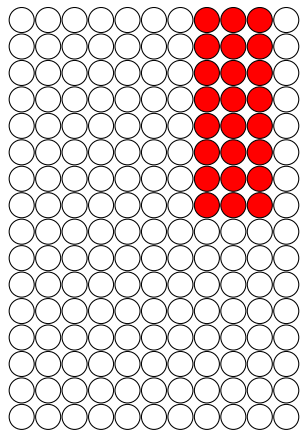
57 digit 9

Font #5 led16a.font

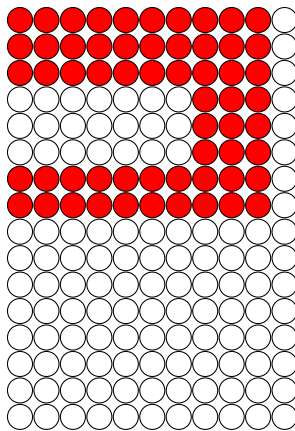
32 space



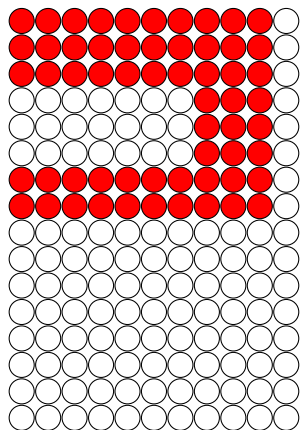
48 digit 0



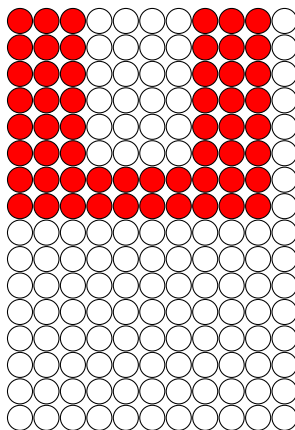
49 digit 1



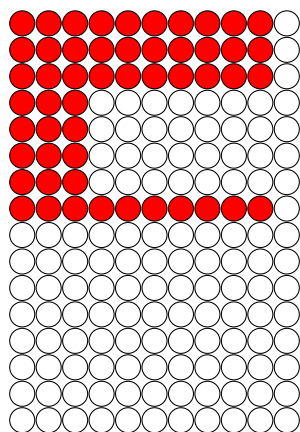
50 digit 2



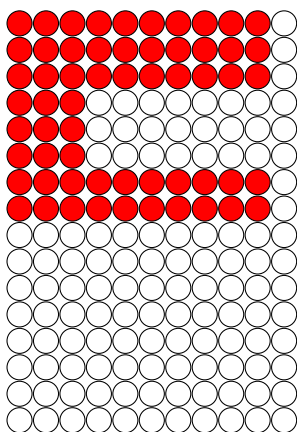
51 digit 3



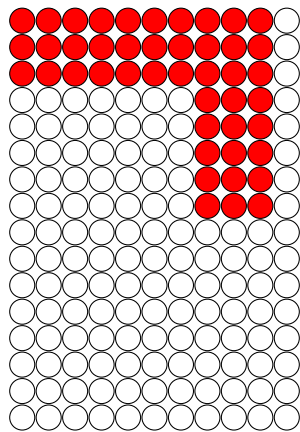
52 digit 4



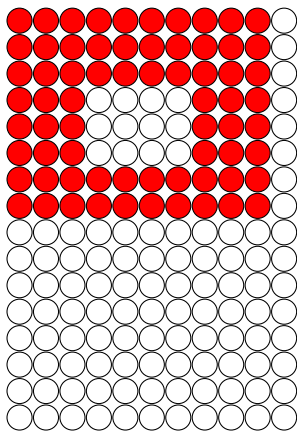
53 digit 5



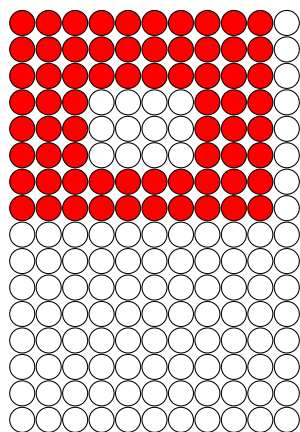
54 digit 6



55 digit 7

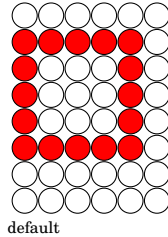


56 digit 8



57 digit 9

Default Glyph



Font Definition Files

In the `firmware/readerboard` directory there are a set of files with `.font` suffixes, which define the fonts that the readerboard can employ when writing text to the matrix. If you edit these files, or add more fonts, make the appropriate adjustments to the `Makefile` and run `make` in that directory to regenerate the glyph images on the preceding pages of this document, and the source files where the glyphs are actually stored in the device firmware. Then recompile the firmware image and flash it onto the hardware devices.

Each file defines up to 256 glyphs with codepoint values 0–255. The firmware will not allow some codepoints to be printed (see above, Tables 2.3, 2.7, and 2.8 on pp. 22–29), so to avoid reserved codes we recommend using codepoints ≥ 32 .

Any line which begins (possibly after leading whitespace) with a semi-colon (`;`) is a comment and is ignored completely.

Each glyph definition begins with a line enclosed in square brackets, in the form

`[<codepoint> <description>]`

Where *<codepoint>* is either the codepoint expressed as a decimal integer (e.g., “42”) or a dollar sign followed by the codepoint as a hex integer (e.g., “\$2B”), and *<description>* is an arbitrary text description of the character being defined.

For example:

```
[ $25 percent]
@@..@|
@@..@|
...@.|
..@..|
.@...|
@..@@|
@..@@|
```

```
[$2E period]
..| | | |
..| | | |
..| | | |
..| | | |
..| | | |
..| | | |
@@| | | |
@@| | | |
```

As a special case, a glyph introduced with the heading

[default]

is defined to be the default glyph to be used if the user tries to print a glyph that is not defined in the font. Only one default glyph may be defined in the entire set of fonts.

The non-blank lines which follow the bracketed header line define the bitmap for the glyph. In this definition, positions which are off are noted with ‘.’ or ‘|’, while positions where the LED is on are noted with ‘@’.

Any completely blank columns to the right of all lit pixels should be represented by ‘|’ rather than ‘.’. This will make note of the drawn width of the character but the columns to the right which consist of ‘|’ characters are not actually stored in the firmware bitmap image, thus conserving ROM space.

If fewer than eight rows are specified, it is assumed that the rows that are present start at the top of the display, and the remaining unspecified rows are all off.

Character Aliases

If a character glyph is to be identical to another glyph, there is no point in storing two or more identical bitmaps in the ROM. In such a case, rather than presenting the duplicate bitmap, the character definition is simply

-><codepoint>
or
-><codepoint>,

In the first case, the glyph for the indicated codepoint will be reused for this character, such as the case in Font #0 where the prime character (hex \$87) looks identical to the single-quote character (hex \$27). Only the single-quote glyph is actually specified in `standard.font`, so the prime character is defined only as

```
[$87 prime]  
->$27
```

In the second case, the glyph to be duplicated comes from a different font file entirely. For example, many characters in Font #1 (`standard_variable.font`) have the same glyphs as Font #0, such as:

```
[$23 pound sign]  
->$23,0
```

PREPARING AND UPDATING FIRMWARE IMAGES

Software is a great combination
between artistry and
engineering.

—Bill Gates

The firmware that runs on the readerboard hardware (or, more precisely, the Arduino single-board computer attached to the circuit board) is located in the `firmware/readerboard` directory of the readerboard project repository.

You'll need the following prerequisites in your Arduino development environment:

- EEPROM v2.0
- I2C_EEPROM v1.8.5
- TimerEvent v0.5.0
- Wire v1.0

After pulling the latest code from the repository if necessary, perform the following steps to configure and prepare the firmware image:

Step 1: Configuration. Edit the `CONFIGURATION` SECTION near the top of `readerboard.h`:

Values for HW_MODEL (Readerboard Hardware Model):	
MODEL_3xx_RGB	PCB revision 3.0 and later with RGB LEDs installed.
MODEL_3xx_MONOCHROME	PCB revision 3.0 and later with single-color LEDs installed with their anodes soldered into pin 4 (the blue circuit).
MODEL_BUSYLIGHT_1	PCB revision 1.x Busylight module.
MODEL_BUSYLIGHT_2	PCB revision 2.x Busylight module.
Values for HW_MC (Microcontroller Installed):	
HW_MC_MEGA_2560	Arduino Mega 2560.
HW_MC_DUE	Arduino Due.
HW_MC_PRO	SparkFun Arduino-compatible Pro Micro.

Table 7.1: Firmware Configuration Values

```
// BEGIN CONFIGURATION SECTION
// TODO: Set these before compiling for a particular hardware
//      configuration
#define HW_MODEL (MODEL_3xx_RGB)
#define HW_MC (HW_MC_DUE)
#define HAS_I2C_EEPROM (false)
```

These values should come already set up for the latest hardware model (in this example, the revision 3 RGB models which incorporate the Arduino directly onto the main circuit board). If your hardware uses a different design, HW_MODEL needs to be changed to another supported hardware name (see Table 7.1).

Also, HW_MC needs to be set to the model of Arduino microcontroller you will be using to control this unit.

If using an EEPROM chip or module external to the Arduino via the I²C bus, such as chip U16 on PCB revision 3.3.0 and later (which is recommended for use with the Arduino Due), then set HAS_I2C_EEPROM to true. If using no EEPROM or the one built-in to the Arduino, leave this set to false.

The supported values for these two configuration settings are listed in Table 7.1.

```
// TODO: Set these default settings (this will be the "factory
//      default settings"). On Due-based systems without
//      EEPROM chip U16 installed, this is the only way to
//      make these settings at all. If the EEPROM is installed
```

```
//      (or one is built-in to the microcontroller), this is
//      just the default that can be overridden by configuring
//      the unit since the new configuration values can be
//      saved in EEPROM.
//
// Default baud rate. Allowed values are '0'=300, '1'=600,
// '2'=1200, '3'=2400, '4'=4800, '5'=9600, '6'=14400,
// '7'=19200, '8'=28800, '9'=31250, 'A'=38400, 'B'=57600,
// 'C'=115200.
#define EE_DEFAULT_USB_SPEED ('5')
#define EE_DEFAULT_485_SPEED ('5')
```

These values determine the default baud rate to use for the USB direct connection and RS-485 network. Note that the value is a character literal.

```
// Default device address (may be any value from 0-63 except
// the global address, or EE_ADDRESS_DISABLED if you won't be
// using RS-485 at all.)
#define EE_DEFAULT_ADDRESS (EE_ADDRESS_DISABLED)
//
// Default global device address (may be any value from 0-15).
#define EE_DEFAULT_GLOBAL_ADDRESS (15)
```

With `EE_DEFAULT_ADDRESS` set to `EE_ADDRESS_DISABLED`, the RS-485 interface will be disabled. Otherwise the value set for this symbol is an integer in the range 0–63 which will be the device address recognized in commands received over the RS-485 network.

Set `EE_DEFAULT_GLOBAL_ADDRESS` to the default global device address you wish the unit to recognize. This must be in the range 0–15.

```
//
// TODO: Adjust these for the colors of discrete status LEDs on
//      this unit. These can be used for color values in
//      commands sent to the device.
//
//      READERBOARDS (8 LIGHTS)
#define R_STATUS_LED_COLOR_L0 ('G')
#define R_STATUS_LED_COLOR_L1 ('Y')
#define R_STATUS_LED_COLOR_L2 ('Y')
#define R_STATUS_LED_COLOR_L3 ('r')
#define R_STATUS_LED_COLOR_L4 ('R')
```

```

#define R_STATUS_LED_COLOR_L5 ('b')
#define R_STATUS_LED_COLOR_L6 ('B')
#define R_STATUS_LED_COLOR_L7 ('W')
//
//      BUSYLIGHTS (7 LIGHTS)
#define B_STATUS_LED_COLOR_L0 ('G')
#define B_STATUS_LED_COLOR_L1 ('y')
#define B_STATUS_LED_COLOR_L2 ('Y')
#define B_STATUS_LED_COLOR_L3 ('r')
#define B_STATUS_LED_COLOR_L4 ('R')
#define B_STATUS_LED_COLOR_L5 ('B')
#define B_STATUS_LED_COLOR_L6 ('W')

```

These are set for the standard color assignments for the discrete status LEDs, allowing for the use of these single-letter color codes to refer to the status LEDs. If you build a unit with different colors, you may assign appropriate color codes here.

Since busylights and readerboards have different numbers of status light displays, they each get a separate set of color code assignments. If you are setting up a nonstandard readerboard, then edit the codes defined here for symbols starting with R_, and likewise with the B_ symbols for nonstandard busylight units.

If your unit doesn't have the full compliment of lights possible, assign the unused positions to codes with a digit that matches the light number. For example, for a three-color busylight:

```

#define B_STATUS_LED_COLOR_L0 ('G') /* green */
#define B_STATUS_LED_COLOR_L1 ('R') /* red */
#define B_STATUS_LED_COLOR_L2 ('B') /* blue */
#define B_STATUS_LED_COLOR_L3 ('O') /* orange */
#define B_STATUS_LED_COLOR_L4 ('4') /* unused */
#define B_STATUS_LED_COLOR_L5 ('5') /* unused */
#define B_STATUS_LED_COLOR_L6 ('6') /* unused */

```

Move on to the version number banners.

```

#define BANNER_HARDWARE_VERS "HW 3.4.1  "
#define BANNER_FIRMWARE_VERS "FW 2.3.8  "
#define SERIAL_VERSION_STAMP "V3.4.1$R2.3.8$"
//
//      \---/  \---/
//      |      |

```



```
//          Hardware version    |
//          Firmware version
```

The `BANNER_FIRMWARE_VERS` value should match the version of the firmware source code you are compiling.

Likewise, `BANNER_HARDWARE_VERS` gives the revision of the PCB in use.

These should be 10-character text strings. They will be displayed briefly when the sign is booted.

Set the `SERIAL_VERSION_STAMP` value to match the previous two values while still preserving the format “V...\$R...\$”. This string will be sent verbatim as part of the device’s response to being queried for its configuration and status.

If, and **only if**, you are making a custom firmware image for a board with no EEPROM storage capability, adjust the `BESPOKE_SERIAL_NUMBER` definition with the serial number of this device. For example:

```
#define BESPOKE_SERIAL_NUMBER "RB1234"
```

Step 2: Make. Run the make program. This will build the font tables from the various *.font files into the firmware code so the target device will be able to generate characters from those fonts.

Customizing and extending the fonts is described in Chapter 6.

Step 3: Compile. Using your Arduino compiler tools, compile the readerboard sketch files and upload the resulting binary to your readerboard hardware. This step assumes you have set up your own Arduino development environment and is beyond the scope of this document to help you with.

Step 4: Serial Number. If your unit has EEPROM memory, run the `setsn` command to field-configure the device’s serial number. For example:

```
% setsn -port /dev/ttyACM0 -sn RB1234 -model rb
```


SCHEMATICS AND DIAGRAMS

Whether you draw diagrams
that generate code or you type at
a browser, you are coding.

—Kent Beck

The following pages include the schematics for the current 3.4.1 version of the readerboard PCB and the board assembly diagram, showing where the components are placed, and the same for the 2.2.0 version of the Busy-light circuit board and 1.1 light tree tier boards for use with the Busylight units.

Two versions of the light tree octagons are provided. Version 1.1 uses through-hole resistors, while version 1.1-S uses surface-mount resistors.

